REFERENCE GUIDE

VC

VIRTUAL CONTROL

VMWARE CLOUD FOUNDATION SOLUTIONS

VCF Undocumented Issues Reference

Real-world undocumented issues, edge cases, and workarounds discovered through hands-on VCF operations not found in official documentation.

EDGE CASES | WORKAROUNDS | UNDOCUMENTED | REAL-WORLD FIXES

VCF 9.0 VMware Cloud Foundation

VCF 9.0 Undocumented Issues & Discoveries Reference

Version: 7.0 | **Date:** May 26, 2026 | **Total Discoveries:** 56 (47 still undocumented + 9 now documented by Broadcom)

> *Every issue in this document was independently verified undocumented by Broadcom as of 2026-05-11. The May 2026 audit removed 9 entries that Broadcom has since covered with KB articles or TechDocs (they remain in `VCF_Troubleshooting_Handbook` for reference) and added 3 new discoveries from the May 2026 esxi02/esxi03 disk-group rebuild work. Each entry includes the exact problem, impact, and **complete copy-paste-ready resolution steps**.*

> ***PowerShell users:*** *Workstation-side `bash` blocks below (REST API calls, OVFTool, vCenter ops) are followed by a paste-ready PowerShell sibling block. For shared idioms (self-signed TLS bypass, `Invoke-RestMethod` patterns, `ConvertFrom-Json`, `curl.exe` usage, here-string handling, etc.) see the central reference: `../POWERSHELL-TRANSLATION-REFERENCE.md`. Appliance-only blocks (psql, systemctl, keytool against `/etc/`, file edits in `/storage/`, `/var/`, `/etc/systemd/network/`) and ESXi-shell blocks (`vmkfstools`, VMX edits) are intentionally not translated — those must run inside their respective shells.*

Table of Contents

1. Summary by Category
2. Database & Credential Operations (7 issues)
3. NSX in Nested/Resource-Constrained Environments (6 issues)
4. Certificate Management (2 issues)
5. VCF Operations 9.x Changes (5 issues)
6. Infrastructure & Platform (2 issues)
7. Crash Recovery & VCF Operations Suite-API (6 issues)
8. Photon OS 5 Networking & OVFTool (2 issues)
9. Aria Suite / LCM / VCF Automation (7 issues)
10. vSAN & DVS Edge Cases — May 2026 (3 issues)
12. NSX 9.0 vDS Replacement / vLCM Lifecycle Edge Cases — May 2026 (3 issues)
13. VCSA Recovery — May 2026 (2 issues)

14. Fleet Management Offline Depot — May 2026 (1 issue)

15. Originally Discovered Undocumented — Now Documented by Broadcom (9 issues)

16. Quick Reference: All 56 Issues

17. Document Information

Where commands run in this issues reference. This catalog covers undocumented edge cases across all VCF tiers. Each issue's commands run from environment(s) noted in the issue's section header — common environments:

- **ESXi host shell (SSH)** — for vSAN, network, host-level fixes.
- **vCenter shell or UI** — for cert ops, service control, inventory recovery.
- **NSX Manager shell or UI** — for transport node, edge, manager cluster fixes.
- **SDDC Manager shell / API** — for workflow recovery, password rotation issues.
- **Workstation (PowerShell / Bash)** — for scripted detection, log harvesting.

Replace any credential placeholder like `<your-password>` with values from your vault.

1. Summary by Category

Category	Issues	Severity Range	Key Impact
Database & Credential Operations	#1–7	Critical	All future credential ops blocked without DB repair
NSX in Nested Environments	#8–13	High	OOM crashes, boot storms, service instability
Certificate Management	#14–15	High	Cert generator wrong SANs, PEM escaping required
VCF Operations 9.x Changes	#16–20	Medium-High	JRE path changed, adapters fail silently
Infrastructure & Platform	#21–22	Medium	vMotion failures, storage waste
Crash Recovery & Suite-API	#23–28	High	Cannot manage VCF Ops without undocumented API formats
Photon OS 5 Networking & OVFTool	#29–30	High	OVFTool silently drops InstanceId props; nmctl absent on VCF Ops
Aria Suite / LCM / VCF Automation	#31–37	Critical	LCM DB cleanup, hidden vmsp entries, vcops-web frontend gate

vSAN & DVS Edge Cases (May 2026)	#38–40	High	vSAN OSA nested misclassification, MM finalization gaps
VCSA Recovery (May 2026)	#41–42	High	Host Client OVA `vAppConfig=null` misreport; VCSA UI Installer cannot use vSAN datastore (KB 389238)
Fleet Management Offline Depot (May 2026)	#47	Critical	Metadata zip references superseded 9.0.2.0 binary; Broadcom only ships 9.0.2.0100; `vm_cr_contentdownloadurl` fix
Now Documented by Broadcom	#48–56	Various	9 issues originally discovered undocumented, now covered by Broadcom KBs/TechDocs

Discovery Timeline:

- Issues #1–15 (excl. NSX): Discovered during initial VCF 9.0.1 deployment (January–February 2026)
- Issues #16–22: Discovered during initial deployment + Windows Update crash recovery (Feb–March 2026)
- Issues #23–28: Discovered during crash recovery (March 2026)
- Issues #29–30: Discovered during VCF Operations redeployment (April 2026)
- Issues #31–37: Discovered during Aria Suite / VCFA deployment (April 14–16, 2026)
- Issues #38–40: Discovered during esxi02/esxi03 vSAN disk-group rebuild (May 9–11, 2026)
- Issues #41–42: Discovered during vCenter recovery + redeploy (May 13, 2026)
- Issue #47: Discovered during Fleet Management offline depot upgrade to 9.0.2.0100 (May 26, 2026)

Audit Note (2026-05-11): Issues renumbered sequentially 1–40 in v6.0 audit. Nine entries that Broadcom has since documented were moved to `VCF_Troubleshooting_Handbook` — see Revision History v6.0 for the mapping of removed entries to current Broadcom KBs.

---

2. Database & Credential Operations

Issue #1: SDDC Manager PostgreSQL Schema Unmapped

Problem: The SDDC Manager PostgreSQL database schema — table names, column names, and relationships — is completely undocumented by Broadcom.

Impact: Cannot troubleshoot credential failures, stuck tasks, or stale locks without knowing the schema.

Resolution — Map the schema yourself:

```
# Step 1: SSH to SDDC Manager
ssh vcf@sddc-manager.lab.local
# Enter password when prompted: <your-password>

# Step 2: Switch to root (needed for postgres access)
su -
```

```
# Enter root password: <your-password>

# Step 3: Connect to PostgreSQL (MUST use -h 127.0.0.1 — see Issue #5)
sudo -u postgres psql -h 127.0.0.1 -d platform

# Step 4: List all tables in the platform database
SELECT table_name FROM information_schema.tables
WHERE table_schema = 'public' ORDER BY table_name;

# Step 5: Get column details for any table
SELECT column_name, data_type, is_nullable
FROM information_schema.columns
WHERE table_name = 'nsxt' ORDER BY ordinal_position;

# Step 6: Check a table's current content
SELECT * FROM nsxt;
SELECT * FROM lock;
SELECT id, status, resolved FROM task_metadata ORDER BY id DESC LIMIT 10;
```

Key tables discovered (complete reference):

Table	Purpose	Key Columns	Notes
`nsxt`	NSX cluster status	`id`, `state` (NOT `status`)	`state` must be `ACTIVE` for credential ops
`lock`	Resource-level locks	`resource_id`, `lock_type`, `created_at`	Stale locks block ALL operations
`task_metadata`	Task tracking	`id`, `resolved` (boolean), `status`	`resolved` must be `true` for completed tasks
`task_lock`	Task-level locks	`task_id`, `resource_id`	Links tasks to locked resources
`credential`	Managed credentials	`id`, `resource_type`, `account_type`	Credential inventory
`host`	ESXi hosts	`id`, `fqdn`, `status`	Commissioned host records

Issue #2: Credential Cascade Failure Mechanism

Problem: A failed credential rotation leaves NSX stuck in `ACTIVATING` state, stale locks accumulate, and unresolved tasks pile up. Each UI retry creates more locks, making it progressively worse.

Impact: All future credential operations are permanently blocked. The SDDC Manager UI shows errors on every credential operation.

How to identify this issue:

```
# bash on workstation (Linux/macOS) or appliance via SSH
# Step 1: Get SDDC Manager API token
TOKEN=$(curl -sk -X POST https://sddc-manager.lab.local/v1/tokens \
  -H "Content-Type: application/json" \
  -d '{"username":"administrator@vsphere.local","password":"<your-password>"}' \
  | python3 -c "import sys,json;print(json.load(sys.stdin)['accessToken'])")

# Step 2: Check for stuck tasks
curl -sk -H "Authorization: Bearer $TOKEN" \
  "https://sddc-manager.lab.local/v1/tasks?status=IN_PROGRESS" | python3 -m
json.tool

# Step 3: Check for resource locks
curl -sk -H "Authorization: Bearer $TOKEN" \
  https://sddc-manager.lab.local/v1/resource-locks | python3 -m json.tool

# Step 4: Check NSX status (should be ACTIVE, not ACTIVATING)
curl -sk -H "Authorization: Bearer $TOKEN" \
  https://sddc-manager.lab.local/v1/nsxt-clusters | python3 -m json.tool
```

PowerShell equivalent:

```
# PowerShell 5.1+ on Windows workstation
[Net.ServicePointManager]::ServerCertificateValidationCallback = { $true }  # PS
5.1 self-signed

# Step 1: Get SDDC Manager API token
$body = @{
    username = 'administrator@vsphere.local'
    password = '<your-password>'
} | ConvertTo-Json
$TOKEN = (Invoke-RestMethod -Method Post `
    -Uri 'https://sddc-manager.lab.local/v1/tokens' `
    -ContentType 'application/json' -Body $body).accessToken
$hdr = @{ Authorization = "Bearer $TOKEN" }

# Step 2: Check for stuck tasks
Invoke-RestMethod -Uri 'https://sddc-manager.lab.local/v1/tasks?
status=IN_PROGRESS' `
    -Headers $hdr | ConvertTo-Json -Depth 6
```

```
# Step 3: Check for resource locks
Invoke-RestMethod -Uri 'https://sddc-manager.lab.local/v1/resource-locks' `
    -Headers $hdr | ConvertTo-Json -Depth 6

# Step 4: Check NSX status (should be ACTIVE, not ACTIVATING)
Invoke-RestMethod -Uri 'https://sddc-manager.lab.local/v1/nsxt-clusters' `
    -Headers $hdr | ConvertTo-Json -Depth 6
```

If you see: stuck IN_PROGRESS tasks, active resource locks, and/or NSX in ACTIVATING state — you have a credential cascade failure. **Follow Issue #4 for the complete 6-step fix.**

Issue #3: API Cannot Cancel Stuck Tasks

Problem: The SDDC Manager API returns `TA_TASK_CAN_NOT_BE_RETRIED` when attempting to retry stuck tasks. DELETE returns HTTP 500.

Impact: Database repair is the only fix path — the API provides no mechanism to resolve this.

How to confirm the API cannot help:

```
# bash on workstation — confirms the documented API is a dead end
# Attempt to retry a stuck task (this will fail)
curl -sk -X PATCH "https://sddc-manager.lab.local/v1/tasks/<task-id>" \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"status":"IN_PROGRESS"}'
# Response: {"errorCode":"TA_TASK_CAN_NOT_BE_RETRIED","message":"Task cannot be
retried"}

# Attempt to delete a stuck task (this will also fail)
curl -sk -X DELETE "https://sddc-manager.lab.local/v1/tasks/<task-id>" \
  -H "Authorization: Bearer $TOKEN"
# Response: HTTP 500
```

PowerShell equivalent:

```
# PowerShell 5.1+ on Windows workstation
[Net.ServicePointManager]::ServerCertificateValidationCallback = { $true }
$hdr = @{ Authorization = "Bearer $TOKEN" }
```

```
# Attempt to retry a stuck task (this will fail)
try {
    Invoke-RestMethod -Method Patch `
        -Uri "https://sddc-manager.lab.local/v1/tasks/<task-id>" `
        -Headers $hdr -ContentType 'application/json' `
        -Body '{"status":"IN_PROGRESS"}'
} catch {
    # Expected: TA_TASK_CAN_NOT_BE_RETRIED
    $_.ErrorDetails.Message
}

# Attempt to delete a stuck task (this will also fail)
try {
    Invoke-RestMethod -Method Delete `
        -Uri "https://sddc-manager.lab.local/v1/tasks/<task-id>" `
        -Headers $hdr
} catch {
    # Expected: HTTP 500
    $_.Exception.Response.StatusCode.value__
}
```

Resolution — Direct SQL fix:

```
# SSH to SDDC Manager and connect to PostgreSQL
ssh vcf@sddc-manager.lab.local
su -
sudo -u postgres psql -h 127.0.0.1 -d platform

# Find the stuck task ID
SELECT id, name, status, resolved FROM task_metadata
WHERE status = 'IN_PROGRESS' OR resolved = false
ORDER BY id DESC;

# Mark the specific stuck task as resolved
UPDATE task_metadata SET resolved = true WHERE id = '<task-id>';

# Or mark ALL unresolved tasks as resolved (nuclear option)
UPDATE task_metadata SET resolved = true WHERE resolved = false;

# Verify the fix
SELECT id, name, status, resolved FROM task_metadata
WHERE resolved = false;
-- Should return 0 rows
```

```
# Exit psql
\q
```

Issue #4: 6-Step PostgreSQL Repair Procedure

Problem: Fixing a credential cascade failure requires updating three tables in a specific sequence. Partial fixes still fail because all three tables participate in prevalidation. You MUST do all steps in order.

Impact: Without all 6 steps in order, the system remains broken.

Complete fix procedure (copy-paste every command):

```
# ================================================================
# SDDC MANAGER CREDENTIAL CASCADE FAILURE — COMPLETE FIX
# ================================================================
# Run each step in order. Do NOT skip steps.
# ================================================================

# STEP 1: SSH to SDDC Manager
ssh vcf@sddc-manager.lab.local
# Password: <your-password>

# Switch to root
su -
# Password: <your-password>

# STEP 2: Enable trust authentication for PostgreSQL
# (Required because the PostgreSQL password is not discoverable)
cp /data/vmware/vcf/commonsvcs/postgresql/pg_hba.conf \
   /data/vmware/vcf/commonsvcs/postgresql/pg_hba.conf.backup

# Change "md5" to "trust" for local connections
sed -i 's/host    all.*127.0.0.1\/32.*md5/host    all             all
127.0.0.1\/32            trust/' \
  /data/vmware/vcf/commonsvcs/postgresql/pg_hba.conf

# Restart PostgreSQL to pick up the change
systemctl restart postgresql

# STEP 3: Connect to the platform database
sudo -u postgres psql -h 127.0.0.1 -d platform
```

```
# STEP 4: Fix NSX cluster status (change ACTIVATING → ACTIVE)
SELECT id, state FROM nsxt;
-- If state shows 'ACTIVATING', run:
UPDATE nsxt SET state = 'ACTIVE' WHERE state = 'ACTIVATING';

# STEP 5: Clear ALL resource locks
SELECT * FROM lock;
-- Note how many rows exist (these are all stale)
DELETE FROM lock;
-- Should return: DELETE <count>

# STEP 6: Mark ALL unresolved tasks as resolved
SELECT id, name, status, resolved FROM task_metadata WHERE resolved = false;
-- Note the stuck tasks
UPDATE task_metadata SET resolved = true WHERE resolved = false;

# STEP 7: Clear task-level locks
SELECT * FROM task_lock;
DELETE FROM task_lock;

# STEP 8: Exit PostgreSQL
\q

# STEP 9: Restore md5 authentication (IMPORTANT — do not leave trust enabled)
cp /data/vmware/vcf/commonsvcs/postgresql/pg_hba.conf.backup \
   /data/vmware/vcf/commonsvcs/postgresql/pg_hba.conf

systemctl restart postgresql

# STEP 10: Restart SDDC Manager operations service
systemctl restart operationsmanager

# Wait 5 minutes for operationsmanager to fully start

# STEP 11: Verify the fix — get a fresh token and check status
TOKEN=$(curl -sk -X POST https://sddc-manager.lab.local/v1/tokens \
  -H "Content-Type: application/json" \
  -d '{"username":"administrator@vsphere.local","password":"<your-password>"}' \
  | python3 -c "import sys,json;print(json.load(sys.stdin)['accessToken'])")

# Should show: no IN_PROGRESS tasks
curl -sk -H "Authorization: Bearer $TOKEN" \
  "https://sddc-manager.lab.local/v1/tasks?status=IN_PROGRESS" \
  | python3 -c "import sys,json;print(len(json.load(sys.stdin).get('elements',
[])),'stuck tasks')"
```

```
# Should show: no resource locks
curl -sk -H "Authorization: Bearer $TOKEN" \
  https://sddc-manager.lab.local/v1/resource-locks \
  | python3 -c "import sys,json;print(len(json.load(sys.stdin).get('elements',
[])),'locks')"

# Should show: NSX status = ACTIVE
curl -sk -H "Authorization: Bearer $TOKEN" \
  https://sddc-manager.lab.local/v1/nsxt-clusters \
  | python3 -c "import sys,json;d=json.load(sys.stdin);print(d['elements'][0]
['status'])"
```

Issue #5: PostgreSQL Requires TCP Connection

Problem: SDDC Manager PostgreSQL doesn't accept Unix socket connections. Must use `-h 127.0.0.1`.

Impact: `psql` without `-h` flag silently fails or connects to the wrong instance.

Resolution:

```
# WRONG — silently fails or connects to system PostgreSQL
sudo -u postgres psql -d platform
# Error: "FATAL: Peer authentication failed" or connects to wrong DB

# CORRECT — always use -h 127.0.0.1
sudo -u postgres psql -h 127.0.0.1 -d platform

# Verify you're connected to the right database
SELECT current_database();
-- Should return: platform

SELECT count(*) FROM nsxt;
-- Should return a row count (1 or more)
```

Issue #6: Database Column Naming Inconsistencies

Problem: The `nsxt` table uses `state` (not `status`), and `task_metadata` uses a `resolved` boolean (not a status enum). If you guess the column names wrong, your fix queries will silently do nothing.

Impact: Wrong column names = wrong queries = no fix.

Resolution — Use the correct column names:

```
-- WRONG — these columns don't exist and will error
UPDATE nsxt SET status = 'ACTIVE';           -- ERROR: column "status" does not
exist
UPDATE task_metadata SET status = 'RESOLVED'; -- This changes the wrong column

-- CORRECT column names
UPDATE nsxt SET state = 'ACTIVE' WHERE state = 'ACTIVATING';
UPDATE task_metadata SET resolved = true WHERE resolved = false;
```

Complete column reference:

Table	Column	Type	Valid Values
`nsxt`	`state`	varchar	`ACTIVE`, `ACTIVATING`, `ERROR`
`task_metadata`	`resolved`	boolean	`true`, `false`
`task_metadata`	`status`	varchar	`SUCCESSFUL`, `FAILED`, `IN_PROGRESS`
`lock`	`resource_id`	varchar	UUID of locked resource

Issue #7: PostgreSQL Password Not Discoverable

Problem: There is no documented method to obtain the SDDC Manager PostgreSQL password. The password is not stored in any accessible config file.

Impact: Cannot connect to the database for troubleshooting without a workaround.

Resolution — Trust authentication workaround:

```
# Step 1: SSH to SDDC Manager as root
ssh vcf@sddc-manager.lab.local
su -

# Step 2: Backup the current pg_hba.conf
cp /data/vmware/vcf/commonsvcs/postgresql/pg_hba.conf \
   /data/vmware/vcf/commonsvcs/postgresql/pg_hba.conf.backup

# Step 3: View current auth settings
cat /data/vmware/vcf/commonsvcs/postgresql/pg_hba.conf
# Look for the line: host  all  all  127.0.0.1/32  md5
```

```
# Step 4: Change md5 to trust for local connections
sed -i 's/host    all.*127.0.0.1\/32.*md5/host    all             all
127.0.0.1\/32            trust/' \
  /data/vmware/vcf/commonsvcs/postgresql/pg_hba.conf

# Step 5: Restart PostgreSQL
systemctl restart postgresql

# Step 6: Now you can connect without a password
sudo -u postgres psql -h 127.0.0.1 -d platform

# Step 7: Do your work...

# Step 8: CRITICAL — Restore md5 auth when done
\q
cp /data/vmware/vcf/commonsvcs/postgresql/pg_hba.conf.backup \
   /data/vmware/vcf/commonsvcs/postgresql/pg_hba.conf
systemctl restart postgresql

# Step 9: Verify auth is restored (this should now fail)
psql -h 127.0.0.1 -U postgres -d platform
# Expected: "FATAL: password authentication failed" — this confirms md5 is back
```

> ***WARNING:*** *Do NOT leave trust authentication enabled. Always restore the backup pg_hba.conf after completing your database work. Trust auth allows anyone with local shell access to connect to the database without credentials.*

---

3. NSX in Nested/Resource-Constrained Environments

Issue #8: NSX Requires 32GB RAM Minimum

Problem: Broadcom documentation states 16GB minimum RAM for NSX Manager. In practice: 16GB = OOM kills, 24GB = intermittent crashes, 32GB = stable.

Impact: Under-provisioned NSX cascades into all VCF operations — credential rotation fails, SDDC Manager reports NSX as "UNSTABLE", and VDT fails.

Resolution — Set correct VM resources:

```
# bash on workstation (Linux/macOS) — vCenter REST API path
# Option A: Via vCenter UI
```

```
# 1. Power off NSX Manager VM
# 2. Right-click > Edit Settings
# 3. Set Memory: 32768 MB (32 GB)
# 4. Set CPU: 6 vCPU
# 5. Power on

# Option B: Via vCenter REST API
SESSION=$(curl -sk -X POST https://vcenter.lab.local/api/session \
  -H "Authorization: Basic $(echo -n 'administrator@vsphere.local:<your-
password>' | base64)" \
  | tr -d '"')

# Power off the NSX Manager VM first
curl -sk -X POST "https://vcenter.lab.local/api/vcenter/vm/vm-58/power?
action=stop" \
  -H "vmware-api-session-id: $SESSION"

# Wait for power off to complete (check status)
curl -sk -H "vmware-api-session-id: $SESSION" \
  "https://vcenter.lab.local/api/vcenter/vm/vm-58" \
  | python3 -c "import sys,json;print(json.load(sys.stdin)['power_state'])"

# Update memory (API requires power off first)
curl -sk -X PATCH "https://vcenter.lab.local/api/vcenter/vm/vm-58" \
  -H "vmware-api-session-id: $SESSION" \
  -H "Content-Type: application/json" \
  -d '{"memory":{"size_MiB":32768}}'

# Power on
curl -sk -X POST "https://vcenter.lab.local/api/vcenter/vm/vm-58/power?
action=start" \
  -H "vmware-api-session-id: $SESSION"
```

PowerShell equivalent:

```
# PowerShell 5.1+ on Windows workstation — vCenter REST API path
# Option A: same UI steps as the bash version above
# Option B: PowerCLI is the most idiomatic Windows path; REST shown for parity

# --- PowerCLI variant (recommended) ---
# Connect-VIServer vcenter.lab.local -User 'administrator@vsphere.local' -Password
'<your-password>'
# $vm = Get-VM -Name 'nsx-manager'
```

```
# Stop-VM -VM $vm -Confirm:$false
# Set-VM -VM $vm -MemoryGB 32 -NumCpu 6 -Confirm:$false
# Start-VM -VM $vm

# --- REST API variant (matches the bash flow) ---
[Net.ServicePointManager]::ServerCertificateValidationCallback = { $true }
$pair  = 'administrator@vsphere.local:<your-password>'
$basic = [Convert]::ToBase64String([Text.Encoding]::UTF8.GetBytes($pair))
$SESSION = (Invoke-RestMethod -Method Post `
    -Uri 'https://vcenter.lab.local/api/session' `
    -Headers @{ Authorization = "Basic $basic" })
$hdr = @{ 'vmware-api-session-id' = $SESSION }

# Power off the NSX Manager VM first
Invoke-RestMethod -Method Post `
    -Uri 'https://vcenter.lab.local/api/vcenter/vm/vm-58/power?action=stop' `
    -Headers $hdr

# Wait for power off to complete
(Invoke-RestMethod -Uri 'https://vcenter.lab.local/api/vcenter/vm/vm-58' `
    -Headers $hdr).power_state

# Update memory (API requires power off first)
$memBody = @{ memory = @{ size_MiB = 32768 } } | ConvertTo-Json
Invoke-RestMethod -Method Patch `
    -Uri 'https://vcenter.lab.local/api/vcenter/vm/vm-58' `
    -Headers $hdr -ContentType 'application/json' -Body $memBody

# Power on
Invoke-RestMethod -Method Post `
    -Uri 'https://vcenter.lab.local/api/vcenter/vm/vm-58/power?action=start' `
    -Headers $hdr
```

Tested configurations:

RAM	vCPU	Result
16 GB	4	OOM kills within 30 minutes
24 GB	6	Intermittent crashes under load
30 GB	6	Stable with occasional high load
32 GB	**6**	**Stable — recommended**

Issue #9: Boot Storm Load >100 is Normal

Problem: After power-on, NSX Manager experiences load averages exceeding 100 on 6 cores for 30-60 minutes. The VIP remains offline until services stabilize.

Impact: Credential operations triggered during boot storms cause cascade failures (Issue #2).

Resolution — Monitor and wait:

```
# SSH to NSX Manager (may take several attempts during boot storm)
ssh admin@192.168.1.71
# Password: <your-password>

# Check load average
get node-stats

# From an ESXi host, monitor NSX VM's CPU
esxtop
# Press 'c' for CPU view, look for the NSX VM process

# Check if VIP is responding (run from any host with network access)
curl -sk --connect-timeout 5 https://nsx-vip.lab.local/api/v1/cluster/status \
  -u admin:'<your-password>' 2>&1 | head -5
# If "Connection refused" or timeout — VIP is still coming up. Wait.

# Check service status from NSX CLI
get service
# Services should all show "running" after 30-60 min
```

PowerShell equivalent (workstation-side VIP probe only — `get node-stats`, `esxtop`, `get service` must stay in their respective NSX/ESXi shells):

```
# PowerShell 5.1+ on Windows workstation
[Net.ServicePointManager]::ServerCertificateValidationCallback = { $true }
$cred = New-Object System.Management.Automation.PSCredential('admin', `
    (ConvertTo-SecureString '<your-password>' -AsPlainText -Force))

# Probe VIP — equivalent to the curl reachability check
try {
    $r = Invoke-RestMethod -Uri 'https://nsx-vip.lab.local/api/v1/cluster/status'
`
        -Credential $cred -TimeoutSec 5
    $r | ConvertTo-Json -Depth 4 | Select-Object -First 5
```

```
} catch {
    "VIP still coming up: $($_.Exception.Message)"
}
```

Timeline after power-on:

Time	Expected State
0-5 min	SSH not responsive, VIP offline
5-15 min	SSH responds, load >50, services starting
15-30 min	Load 10-50, some services running
30-60 min	Load <10, all services stable, VIP online

> ***Rule:*** *Do NOT run credential rotations, certificate operations, or SDDC Manager tasks until load average drops below 10 and VIP is responding.*

Issue #10: Adding More vCPU is Counterproductive

Problem: Increasing NSX Manager vCPU count beyond 6 causes worse performance due to VMware co-scheduling overhead. ESXi must schedule all vCPUs simultaneously, meaning more vCPUs = harder to schedule = more wait time.

Impact: The intuitive fix (more CPU) actually makes the problem worse.

Resolution:

```
# If NSX Manager has >6 vCPU, reduce it:
# 1. Power off the NSX Manager VM
# 2. In vCenter UI: Right-click > Edit Settings
# 3. Set CPU to 6 vCPU
# 4. Power on

# If performance is still poor with 6 vCPU + 32GB RAM:
# Increase RAM to 48GB instead of adding more CPU
# The bottleneck is memory pressure from Java/Corfu, not CPU
```

Issue #11: Services Take 10-15 Minutes to Stabilize

Problem: After NSX Manager restart, services take 10-15 minutes to fully stabilize. The API returns `error 101` during this period.

Impact: Premature API calls fail and can trigger unnecessary retries. SDDC Manager may interpret the errors as a real failure and start cascading.

Resolution — Wait and verify:

```
# bash on workstation — polling loop
# After restarting NSX Manager, run this loop from any machine with access:
for i in $(seq 1 30); do
  STATUS=$(curl -sk --connect-timeout 5 \
    -u admin:'<your-password>' \
    https://nsx-vip.lab.local/api/v1/cluster/status 2>/dev/null \
    | python3 -c "import
sys,json;d=json.load(sys.stdin);print(d.get('control_cluster_status',
{}).get('status','UNAVAILABLE'))" 2>/dev/null || echo "UNREACHABLE")
  echo "[$(date +%H:%M:%S)] Cluster status: $STATUS"
  if [ "$STATUS" = "STABLE" ]; then
    echo "NSX Manager is ready."
    break
  fi
  sleep 30
done
```

PowerShell equivalent:

```
# PowerShell 5.1+ on Windows workstation
[Net.ServicePointManager]::ServerCertificateValidationCallback = { $true }
$cred = New-Object System.Management.Automation.PSCredential('admin', `
    (ConvertTo-SecureString '<your-password>' -AsPlainText -Force))

for ($i = 1; $i -le 30; $i++) {
    $STATUS = 'UNREACHABLE'
    try {
        $r = Invoke-RestMethod -Uri 'https://nsx-
vip.lab.local/api/v1/cluster/status' `
            -Credential $cred -TimeoutSec 5
        if ($r.control_cluster_status -and $r.control_cluster_status.status) {
            $STATUS = $r.control_cluster_status.status
        } else {
            $STATUS = 'UNAVAILABLE'
        }
    } catch { $STATUS = 'UNREACHABLE' }

    "[{0}] Cluster status: {1}" -f (Get-Date -Format HH:mm:ss), $STATUS
```

```
    if ($STATUS -eq 'STABLE') {
        'NSX Manager is ready.'
        break
    }
    Start-Sleep -Seconds 30
}
```

Issue #12: DNS/NTP Must Be Set via Admin CLI

Problem: DNS and NTP settings must be configured using the NSX admin CLI, not the UI. UI settings don't persist in some nested configurations.

Impact: DNS resolution fails, causing certificate validation errors, SDDC Manager communication failures, and VDT failures.

Resolution — Set via CLI:

```
# SSH to NSX Manager
ssh admin@nsx-vip.lab.local
# Password: <your-password>

# Set DNS server(s)
set name-servers 192.168.1.230

# Set NTP server(s)
set ntp-servers 192.168.1.230

# Verify DNS
nslookup vcenter.lab.local
nslookup sddc-manager.lab.local

# Verify NTP
get ntp-servers
get ntp-status
```

Issue #13: TEP on vmk0 (New in NSX 9.0)

Problem: NSX 9.0 supports "Use VMkernel Adapter" which reuses vmk0 as a Tunnel Endpoint (TEP), eliminating the need for a dedicated TEP VLAN. This is new in NSX 9.0 and not clearly documented.

Impact: Simplifies nested environments — no dedicated TEP VLAN required.

Resolution — Configure during Transport Node Profile creation:

```
In NSX Manager UI:
1. Navigate to: System > Fabric > Profiles > Transport Node Profiles
2. Click "Add Profile"
3. Under "Host Switch" configuration:
   - Type: VDS (Virtual Distributed Switch)
   - Mode: Standard
4. Under "IP Assignment":
   - Select: "Use VMkernel Adapter"
   - Select: vmk0
5. This reuses the management VMkernel as the TEP interface
6. No additional VLAN or IP pool configuration needed
```

> ***Note:*** *This is only recommended for nested lab environments or environments where a dedicated TEP VLAN is not available. Production environments should use a dedicated TEP VLAN for performance isolation.*

---

4. Certificate Management

Issue #14: VCF Ops for Logs Cert Generator — Same SAN Mismatch

Problem: Identical pattern to the Fleet Management cert generator gotcha (now documented by Broadcom in KB 412322 / KB 421072 and moved to `VCF_Troubleshooting_Handbook`) — the VCF Operations for Logs certificate generator produces wrong SANs.

Impact: Logs deployment wizard precheck fails with the same "hosts in the certificate doesn't match" error.

Resolution — Generate a correct certificate manually with OpenSSL:

```
# Run on SDDC Manager (SSH as root) or any Linux host with openssl

# STEP 1: Create OpenSSL config for VCF Ops for Logs
# Replace logs.lab.local / 192.168.1.242 with your Logs FQDN/IP
cat > /tmp/vrli-cert.cnf << 'EOF'
[req]
default_bits = 4096
prompt = no
default_md = sha256
distinguished_name = dn
req_extensions = v3_req
```

```
x509_extensions = v3_req

[dn]
C = US
ST = California
L = Lab
O = Lab
OU = VCF
CN = logs.lab.local

[v3_req]
basicConstraints = CA:FALSE
keyUsage = digitalSignature, keyEncipherment
extendedKeyUsage = serverAuth, clientAuth
subjectAltName = @alt_names

[alt_names]
DNS.1 = logs.lab.local
DNS.2 = logs
IP.1 = 192.168.1.242
EOF

# STEP 2: Generate certificate and key
openssl req -x509 -nodes -days 730 -newkey rsa:4096 \
  -keyout /tmp/vrli.key -out /tmp/vrli.crt \
  -config /tmp/vrli-cert.cnf

# STEP 3: Verify SANs are correct
openssl x509 -in /tmp/vrli.crt -noout -text | grep -A5 "Subject Alternative Name"
# Expected: DNS:logs.lab.local, DNS:logs, IP Address:192.168.1.242

# STEP 4: Display cert and key for copy-paste into the wizard
echo "=== CERTIFICATE ==="
cat /tmp/vrli.crt
echo ""
echo "=== PRIVATE KEY ==="
cat /tmp/vrli.key

# STEP 5: In the Logs deployment wizard:
# 1. At the "Certificate" step, select "Import"
# 2. Paste the certificate (vrli.crt) and key (vrli.key)
# 3. Click "Validate"
# 4. Run Precheck — should now pass

# STEP 6: After deployment, verify the cert on the deployed appliance
```

```
openssl s_client -connect logs.lab.local:443 -servername logs.lab.local \
  </dev/null 2>/dev/null | openssl x509 -noout -subject -issuer -dates
```

Issue #15: Shell Can't Handle PEM Escaping for NSX API

Problem: When importing certificates via the NSX REST API, the JSON payload requires PEM certificate content with proper newline escaping. Bash/curl with inline PEM content breaks because PEM files contain newlines that JSON requires escaped as `\n`.

Impact: Cannot import certificates via a simple curl command.

Resolution — Use Python to construct and send the JSON payload:

```
# bash on appliance OR workstation (paths shown for appliance: /tmp/nsx.crt)
# OPTION A: Python script (recommended)
python3 << 'PYEOF'
import json, requests, urllib3
urllib3.disable_warnings()

# Read cert and key files
with open('/tmp/nsx.crt') as f:
    cert = f.read()
with open('/tmp/nsx.key') as f:
    key = f.read()

# Construct JSON payload — Python handles the escaping automatically
payload = {"pem_encoded": cert, "private_key": key}

# Import certificate
resp = requests.post(
    "https://192.168.1.71/api/v1/trust-management/certificates?action=import",
    auth=("admin", "<your-password>"),
    json=payload,
    verify=False
)
print(f"Status: {resp.status_code}")
print(f"Response: {resp.text}")
# Save the certificate ID from the response
PYEOF

# OPTION B: Create JSON file first, then use curl
python3 -c "
import json
```

```
cert = open('/tmp/nsx.crt').read()
key = open('/tmp/nsx.key').read()
print(json.dumps({'pem_encoded': cert, 'private_key': key}))
" > /tmp/nsx-import.json

# Then use curl with the JSON file
curl -k -u admin:'<your-password>' \
  -X POST "https://192.168.1.71/api/v1/trust-management/certificates?
action=import" \
  -H "Content-Type: application/json" \
  -d @/tmp/nsx-import.json
```

PowerShell equivalent (workstation-side, with cert/key already copied to e.g. `C:\temp\`):

```
# PowerShell 5.1+ on Windows workstation
# `Invoke-RestMethod` + `ConvertTo-Json` handles PEM newline escaping natively;
# no Python needed.
[Net.ServicePointManager]::ServerCertificateValidationCallback = { $true }

$cert = Get-Content -Raw 'C:\temp\nsx.crt'
$key  = Get-Content -Raw 'C:\temp\nsx.key'
$body = @{
    pem_encoded = $cert
    private_key = $key
} | ConvertTo-Json

$cred = New-Object System.Management.Automation.PSCredential('admin', `
    (ConvertTo-SecureString '<your-password>' -AsPlainText -Force))

$resp = Invoke-RestMethod -Method Post `
    -Uri 'https://192.168.1.71/api/v1/trust-management/certificates?action=import'
`
    -Credential $cred -ContentType 'application/json' -Body $body
$resp | Format-List       # save the certificate id from the response
```

Why this matters: *If you try to embed PEM content directly in a curl `-d` argument, the newlines in the PEM file break the JSON structure. Python's `json.dumps()` and PowerShell's `ConvertTo-Json` both properly escape `\n` characters automatically.*

---

5. VCF Operations 9.x Changes

Issue #16: JRE Path Changed

Problem: The JRE path changed to `/usr/java/jre-vmware-17/`. The legacy `jre-vmware` symlink doesn't exist.

Impact: Cannot import certificates into the correct truststore; keytool commands fail.

Resolution — Use the correct JRE path:

```
# SSH to VCF Operations node
ssh root@192.168.1.77

# CORRECT JRE path in VCF Ops 9.x
ls /usr/java/jre-vmware-17/

# CORRECT cacerts path
ls -la /usr/java/jre-vmware-17/lib/security/cacerts

# Import a CA certificate into the correct truststore
keytool -import -trustcacerts -alias my-ca \
  -file /tmp/ca-cert.pem \
  -keystore /usr/java/jre-vmware-17/lib/security/cacerts \
  -storepass changeit -noprompt

# List certificates in the truststore
keytool -list -keystore /usr/java/jre-vmware-17/lib/security/cacerts \
  -storepass changeit | grep my-ca

# WRONG legacy path (does not exist)
ls /usr/java/jre-vmware/
# Error: No such file or directory
```

Issue #17: Two Separate NSX Adapters

Problem: VCF Operations has two separate NSX adapters — the VCF section auto-creates one using the VIP, while the "Aria Admin" section uses the individual node FQDN.

Impact: Both need separate credentials configured. The Aria Admin adapter may continue working when the VIP is down.

Resolution — Configure both adapters:

```
# bash on workstation — Suite-API REST flow
# Get Suite-API token
TOKEN=$(curl -sk -X POST https://192.168.1.77/suite-api/api/auth/token/acquire \
  -H "Content-Type: application/json" \
  -d '{"username":"admin","password":"<your-password>","authSource":"local"}' \
  | python3 -c "import sys,json;print(json.load(sys.stdin)['token'])")

# List all adapters to see both NSX adapters
curl -sk -H "Authorization: vRealizeOpsToken $TOKEN" \
  https://192.168.1.77/suite-api/api/adapters \
  | python3 -c "
import sys,json
d=json.load(sys.stdin)
for a in d.get('adapterInstancesInfoDto',[]):
    kind=a.get('resourceKey',{}).get('adapterKindKey','')
    if 'NSX' in kind.upper():
        name=a.get('resourceKey',{}).get('name','?')
        aid=a.get('id','?')
        print(f'ID: {aid} | Name: {name} | Kind: {kind}')
"

# You will see two NSX-related adapters:
# 1. VCF adapter's auto-discovered NSX (uses VIP: nsx-vip.lab.local)
# 2. Standalone NSXTAdapter (may use node FQDN: nsx-manager.lab.local)
# Both need valid credentials to function properly
```

PowerShell equivalent:

```
# PowerShell 5.1+ on Windows workstation
[Net.ServicePointManager]::ServerCertificateValidationCallback = { $true }

# Get Suite-API token (legacy header — see Issue #37 for OpsToken)
$body = @{
    username   = 'admin'
    password   = '<your-password>'
    authSource = 'local'
} | ConvertTo-Json
$TOKEN = (Invoke-RestMethod -Method Post `
    -Uri 'https://192.168.1.77/suite-api/api/auth/token/acquire' `
    -ContentType 'application/json' -Body $body).token
```

```
$hdr = @{ Authorization = "vRealizeOpsToken $TOKEN"; Accept = 'application/json'
}

# List all adapters and filter for NSX-related kinds
$adapters = Invoke-RestMethod -Uri 'https://192.168.1.77/suite-api/api/adapters'
`
    -Headers $hdr
$adapters.adapterInstancesInfoDto |
    Where-Object { $_.resourceKey.adapterKindKey -match 'NSX' } |
    ForEach-Object {
        "ID: $($_.id) | Name: $($_.resourceKey.name) | Kind:
$($_.resourceKey.adapterKindKey)"
    }
```

Issue #18: System Managed Credential ROTATE Doesn't Work for NSX

Problem: The system-managed credential rotation for the NSX adapter silently fails — the credential is not actually rotated, but no error is shown.

Impact: NSX monitoring stops when the password changes but the adapter still has the old password.

Resolution — Set credentials manually:

```
In VCF Operations Admin UI (https://192.168.1.77/):
1. Navigate to: Administration > Solutions > Adapters
2. Find the NSX adapter (NSXTAdapter)
3. Click the adapter name to edit
4. Under Credential:
   - UNCHECK "System Managed"
   - Enter the username: admin
   - Enter the current password manually
5. Click Save
6. Click "Test Connection" to verify
7. If the connection test passes, the adapter will resume data collection
```

Issue #19: SSH Enable via Admin UI Only

Problem: SSH access to VCF Operations can only be enabled through the Admin UI at `https://<vcf-ops>:443/admin/`. Console and systemctl approaches don't work.

Impact: Cannot SSH for troubleshooting without Admin UI access first.

Resolution:

```
1. Open a browser and navigate to: https://192.168.1.77/admin/
2. Log in with:
   - Username: admin
   - Password: <your-password>
3. Navigate to: Administration > Access > SSH
4. Toggle SSH to: Enabled
5. Click Save

# Now you can SSH:
ssh root@192.168.1.77
# Password: <your-password>
```

> ***If Admin UI is not accessible:*** *You must use the VM console (vCenter > VM > Launch Console) to access the appliance. From the console, the* `admin` *user can access the Admin UI settings.*

Issue #20: Health Adapter Silently Fails on Stale Credential

Problem: The VMWARE_INFRA_HEALTH adapter silently fails when an SDDC Manager credential becomes stale (e.g., after password rotation). The UI stop/start does not fix it.

Impact: Health monitoring stops. No alerts, no health data collection.

Resolution — Full appliance reboot required:

```
# bash on workstation — Steps 1-2 are workstation-side; Step 3 SSH is appliance-
side
# Step 1: Verify the health adapter is actually failing
TOKEN=$(curl -sk -X POST https://192.168.1.77/suite-api/api/auth/token/acquire \
  -H "Content-Type: application/json" \
  -d '{"username":"admin","password":"<your-password>","authSource":"local"}' \
  | python3 -c "import sys,json;print(json.load(sys.stdin)['token'])")

curl -sk -H "Authorization: vRealizeOpsToken $TOKEN" \
  https://192.168.1.77/suite-api/api/adapters \
  | python3 -c "
import sys,json
d=json.load(sys.stdin)
for a in d.get('adapterInstancesInfoDto',[]):
    kind=a.get('resourceKey',{}).get('adapterKindKey','')
    if 'HEALTH' in kind:
```

```
        name=a.get('resourceKey',{}).get('name','?')
        status=a.get('adapter-status',{}).get('adapterStatus','?')
        print(f'{name}: {status}')
"

# Step 2: Try stop/start first (usually doesn't work but worth trying)
# Find the adapter ID from the output above, then:
curl -sk -X PUT \
  "https://192.168.1.77/suite-api/api/adapters/<adapter-id>/monitoringstate/stop"
\
  -H "Authorization: vRealizeOpsToken $TOKEN"
sleep 10
curl -sk -X PUT \
  "https://192.168.1.77/suite-api/api/adapters/<adapter-id>/monitoringstate/start"
\
  -H "Authorization: vRealizeOpsToken $TOKEN"

# Step 3: If stop/start doesn't fix it, reboot the appliance
ssh root@192.168.1.77
reboot

# Wait 10-15 minutes for the appliance to fully restart
# Then verify the health adapter is collecting data again
```

PowerShell equivalent (workstation-side Steps 1 and 2 only — Step 3 `reboot` must run inside the appliance SSH session):

```
# PowerShell 5.1+ on Windows workstation
[Net.ServicePointManager]::ServerCertificateValidationCallback = { $true }

# Step 1: Verify the health adapter is actually failing
$body = @{
    username = 'admin'; password = '<your-password>'; authSource = 'local'
} | ConvertTo-Json
$TOKEN = (Invoke-RestMethod -Method Post `
    -Uri 'https://192.168.1.77/suite-api/api/auth/token/acquire' `
    -ContentType 'application/json' -Body $body).token
$hdr = @{ Authorization = "vRealizeOpsToken $TOKEN"; Accept = 'application/json'
}

$adapters = Invoke-RestMethod -Uri 'https://192.168.1.77/suite-api/api/adapters'
-Headers $hdr
$adapters.adapterInstancesInfoDto |
```

```
    Where-Object { $_.resourceKey.adapterKindKey -match 'HEALTH' } |
    ForEach-Object {
        "$($_.resourceKey.name): $($_.'adapter-status'.adapterStatus)"
    }

# Step 2: Try stop/start first (usually doesn't work but worth trying)
$adapterId = '<adapter-id>'
Invoke-RestMethod -Method Put `
    -Uri "https://192.168.1.77/suite-
api/api/adapters/$adapterId/monitoringstate/stop" `
    -Headers $hdr
Start-Sleep -Seconds 10
Invoke-RestMethod -Method Put `
    -Uri "https://192.168.1.77/suite-
api/api/adapters/$adapterId/monitoringstate/start" `
    -Headers $hdr

# Step 3: appliance reboot — run inside the SSH session as shown in the bash block
```

---

6. Infrastructure & Platform

Issue #21: `vhv.enable` Ghost Setting

Problem: The `vhv.enable` setting persists in a VM's runtime DICT (vmware.log) even when it is not present in the VMX file. This causes vMotion to fail with: "*The virtual machine cannot be restored because the snapshot was taken with VHV enabled.*"

Impact: vMotion fails with a confusing error. The vCenter UI shows "Expose hardware assisted virtualization" unchecked, and the VMX file has no `vhv.enable` entry — yet the setting is active.

Resolution — Explicitly set FALSE in the VMX file:

```
# Step 1: Power off the VM

# Step 2: SSH to the ESXi host where the VM resides
ssh root@192.168.1.201

# Step 3: Check if vhv.enable exists in the VMX file
grep -i vhv /vmfs/volumes/vcenter-cl01-ds-vsan01/sddc-manager/sddc-manager.vmx
# If no output, the setting is NOT in the file (but may be in runtime)
```

```
# Step 4: Add explicit FALSE — even if the line doesn't exist
echo 'vhv.enable = "FALSE"' >> /vmfs/volumes/vcenter-cl01-ds-vsan01/sddc-
manager/sddc-manager.vmx

# Step 5: Verify it was added
grep -i vhv /vmfs/volumes/vcenter-cl01-ds-vsan01/sddc-manager/sddc-manager.vmx
# Should show: vhv.enable = "FALSE"

# Step 6: Power on the VM and retry vMotion
```

Key lesson: *The ABSENCE of* `vhv.enable` *in the VMX file does NOT mean it is disabled. The setting can persist from a previous deployment environment. You must always add an explicit* `vhv.enable = "FALSE"` *to fix vMotion failures.*

Issue #22: Hot vMotion Fails in Nested Environments

Problem: Live vMotion (hot migration) fails due to memory convergence timeout. The hypervisor cannot converge the memory pages fast enough through the nested network stack.

Impact: Must use cold migration as a fallback. This means VM downtime during migration.

Resolution — Use cold migration:

```
# bash on workstation — vCenter REST API
# Step 1: Power off the VM
# Via vCenter UI: Right-click VM > Power > Shut Down Guest OS
# Or via API:
SESSION=$(curl -sk -X POST https://vcenter.lab.local/api/session \
  -H "Authorization: Basic $(echo -n 'administrator@vsphere.local:<your-
password>' | base64)" \
  | tr -d '"')
curl -sk -X POST "https://vcenter.lab.local/api/vcenter/vm/<vm-id>/power?
action=stop" \
  -H "vmware-api-session-id: $SESSION"

# Step 2: In vCenter UI:
# Right-click VM > Migrate
# Select "Change both compute resource and storage"
# Select destination host/datastore
# The migration will proceed as a cold migration (relocate powered-off VM)

# Step 3: Power on VM at the new location
```

```
curl -sk -X POST "https://vcenter.lab.local/api/vcenter/vm/<vm-id>/power?
action=start" \
  -H "vmware-api-session-id: $SESSION"
```

PowerShell equivalent:

```
# PowerShell 5.1+ on Windows workstation
# PowerCLI variant is the most idiomatic Windows path:
#   Connect-VIServer vcenter.lab.local -User 'administrator@vsphere.local' -
Password '<your-password>'
#   $vm = Get-VM -Name '<vm-name>'
#   Stop-VM -VM $vm -Confirm:$false
#   Move-VM -VM $vm -Destination (Get-VMHost <dst-host>) -Datastore (Get-Datastore
<dst-ds>)
#   Start-VM -VM $vm

# REST API variant (matches the bash flow):
[Net.ServicePointManager]::ServerCertificateValidationCallback = { $true }
$pair  = 'administrator@vsphere.local:<your-password>'
$basic = [Convert]::ToBase64String([Text.Encoding]::UTF8.GetBytes($pair))
$SESSION = (Invoke-RestMethod -Method Post `
    -Uri 'https://vcenter.lab.local/api/session' `
    -Headers @{ Authorization = "Basic $basic" })
$hdr = @{ 'vmware-api-session-id' = $SESSION }

# Step 1: Power off
Invoke-RestMethod -Method Post `
    -Uri 'https://vcenter.lab.local/api/vcenter/vm/<vm-id>/power?action=stop' `
    -Headers $hdr

# Step 2: cold-migrate via vCenter UI as described in bash block

# Step 3: Power on at new location
Invoke-RestMethod -Method Post `
    -Uri 'https://vcenter.lab.local/api/vcenter/vm/<vm-id>/power?action=start' `
    -Headers $hdr
```

Alternative: *If you must avoid downtime, try increasing the vMotion timeout in the advanced settings:* `Host > Configuration > Advanced Settings > Migrate.Enabled = 1` *and* `Migrate.PreCopyAbsoluteMaxRound = 200` *. This may help in some cases but is not guaranteed for nested environments.*

---

7. Crash Recovery & VCF Operations Suite-API

These issues were discovered during the Windows Update crash recovery in March 2026.

Issue #23: VCF Operations Permissions API Requires Single Object (Not Array)

Problem: The PUT body for user permissions at `/suite-api/api/auth/users/{id}/permissions` must be a single JSON object with `roleName`, NOT wrapped in an array, `permissions` key, or any other wrapper.

Impact: Every other format returns "Role with name: null cannot be found" — an unhelpful error that doesn't indicate the format is wrong.

Resolution — Complete working example:

```
# bash on workstation — Suite-API REST flow ($TOKEN from prior step)
# STEP 1: Get the user ID you want to modify
curl -sk -H "Authorization: vRealizeOpsToken $TOKEN" \
  https://192.168.1.77/suite-api/api/auth/users \
  | python3 -c "
import sys,json
d=json.load(sys.stdin)
for u in d.get('users',[]):
    print(f\"ID: {u['id']} | Username: {u['username']} | Roles:
{u.get('roleNames',[])}\")"

# STEP 2: List available roles
curl -sk -H "Authorization: vRealizeOpsToken $TOKEN" \
  https://192.168.1.77/suite-api/api/auth/roles \
  | python3 -c "
import sys,json
d=json.load(sys.stdin)
for r in d.get('roles',[]):
    print(f\"Role: {r['name']} | Privileges: {len(r.get('privilege-keys',[]))}\")"

# STEP 3: Assign the Administrator role to a user
# Replace <USER-ID> with the actual user ID from step 1

# CORRECT format — single JSON object, NOT an array:
curl -sk -X PUT \
  "https://192.168.1.77/suite-api/api/auth/users/<USER-ID>/permissions" \
  -H "Authorization: vRealizeOpsToken $TOKEN" \
  -H "Content-Type: application/json" \
```

```
  -H "Accept: application/json" \
  -d '{"roleName":"Administrator","allowAllObjects":true,"traversal-spec-
instances":[]}'

# WRONG formats that all return "Role with name: null":
# {"permissions": [{"roleName": "Administrator"}]}
# [{"roleName": "Administrator"}]
# {"roleName": "Administrator", "permissions": []}
# {"role": {"name": "Administrator"}}

# STEP 4: Verify the role was assigned
curl -sk -H "Authorization: vRealizeOpsToken $TOKEN" \
  "https://192.168.1.77/suite-api/api/auth/users/<USER-ID>" \
  | python3 -c "import
sys,json;d=json.load(sys.stdin);print('Roles:',d.get('roleNames',[]))"
```

PowerShell equivalent:

```
# PowerShell 5.1+ on Windows workstation ($TOKEN, $hdr from prior step)
[Net.ServicePointManager]::ServerCertificateValidationCallback = { $true }
$hdr = @{ Authorization = "vRealizeOpsToken $TOKEN"; Accept = 'application/json'
}

# STEP 1: Get the user ID you want to modify
$users = Invoke-RestMethod -Uri 'https://192.168.1.77/suite-api/api/auth/users' -
Headers $hdr
$users.users | ForEach-Object {
    "ID: $($_.id) | Username: $($_.username) | Roles: $($_.roleNames -join ',')"
}

# STEP 2: List available roles
$roles = Invoke-RestMethod -Uri 'https://192.168.1.77/suite-api/api/auth/roles' -
Headers $hdr
$roles.roles | ForEach-Object {
    "Role: $($_.name) | Privileges: $($_.'privilege-keys'.Count)"
}

# STEP 3: Assign the Administrator role to a user
$USER_ID = '<USER-ID>'
# CORRECT format — single JSON object, NOT an array:
$permBody = @{
    roleName                = 'Administrator'
    allowAllObjects         = $true
```

```
    'traversal-spec-instances' = @()
} | ConvertTo-Json
Invoke-RestMethod -Method Put `
    -Uri "https://192.168.1.77/suite-api/api/auth/users/$USER_ID/permissions" `
    -Headers $hdr -ContentType 'application/json' -Body $permBody

# WRONG formats that all return "Role with name: null":
#   @{ permissions = @(@{ roleName = 'Administrator' }) } | ConvertTo-Json
#   @(@{ roleName = 'Administrator' }) | ConvertTo-Json
#   @{ roleName = 'Administrator'; permissions = @() } | ConvertTo-Json
#   @{ role = @{ name = 'Administrator' } } | ConvertTo-Json

# STEP 4: Verify the role was assigned
(Invoke-RestMethod -Uri "https://192.168.1.77/suite-api/api/auth/users/$USER_ID"
`
    -Headers $hdr).roleNames
```

Issue #24: VCF Operations Super Admin Shows Empty Roles (By Design)

Problem: The built-in `admin` user always shows `roleNames: []` in the Suite-API. This looks like a bug but is by design.

Impact: Administrators waste time trying to "fix" the admin role assignment. Any attempt to modify the admin user fails.

Resolution — No action needed. Confirm it's working:

```
# bash on workstation — Suite-API REST flow
# Verify admin has implicit full access despite empty roles
TOKEN=$(curl -sk -X POST https://192.168.1.77/suite-api/api/auth/token/acquire \
  -H "Content-Type: application/json" \
  -d '{"username":"admin","password":"<your-password>","authSource":"local"}' \
  | python3 -c "import sys,json;print(json.load(sys.stdin)['token'])")

# These all work — proving admin has full access:
echo "Users (admin-only):"
curl -sk -H "Authorization: vRealizeOpsToken $TOKEN" \
  https://192.168.1.77/suite-api/api/auth/users -o /dev/null -w "%{http_code}\n"

echo "Adapters:"
curl -sk -H "Authorization: vRealizeOpsToken $TOKEN" \
  https://192.168.1.77/suite-api/api/adapters -o /dev/null -w "%{http_code}\n"
```

```
echo "Cluster status:"
curl -sk -H "Authorization: vRealizeOpsToken $TOKEN" \
  https://192.168.1.77/suite-api/api/deployment/node/status -o /dev/null -w "%
{http_code}\n"
# All should return: 200

# These will FAIL (by design):
# PUT to modify admin user → HTTP 500 "Cannot create or update super admin"
# DELETE admin user → HTTP 500 "system created and cannot be deleted"
# Neither of these is a bug — admin is a protected super admin account
```

PowerShell equivalent:

```
# PowerShell 5.1+ on Windows workstation
[Net.ServicePointManager]::ServerCertificateValidationCallback = { $true }

# Verify admin has implicit full access despite empty roles
$body = @{
    username = 'admin'; password = '<your-password>'; authSource = 'local'
} | ConvertTo-Json
$TOKEN = (Invoke-RestMethod -Method Post `
    -Uri 'https://192.168.1.77/suite-api/api/auth/token/acquire' `
    -ContentType 'application/json' -Body $body).token
$hdr = @{ Authorization = "vRealizeOpsToken $TOKEN"; Accept = 'application/json'
}

# Helper to print HTTP status code (Invoke-WebRequest exposes StatusCode directly)
function Get-Status($url) {
    try {
        (Invoke-WebRequest -Uri $url -Headers $hdr -UseBasicParsing).StatusCode
    } catch {
        $_.Exception.Response.StatusCode.value__
    }
}

"Users (admin-only): $(Get-Status 'https://192.168.1.77/suite-
api/api/auth/users')"
"Adapters:          $(Get-Status 'https://192.168.1.77/suite-api/api/adapters')"
"Cluster status:    $(Get-Status 'https://192.168.1.77/suite-
api/api/deployment/node/status')"
# All should return: 200

# These will FAIL (by design):
```

```
#   Invoke-RestMethod -Method Put    -Uri '.../api/auth/users/admin' -Headers $hdr
-Body ... → 500
#   Invoke-RestMethod -Method Delete -Uri '.../api/auth/users/admin' -Headers $hdr
→ 500
```

Issue #25: SDDC Manager domainmanager Uses HTTP (Not HTTPS) on Port 7200

Problem: The `domainmanager` service on SDDC Manager listens on port 7200 using plain **HTTP** (not HTTPS). Using `curl -sk https://localhost:7200` fails with "wrong version number" — a confusing error that suggests a TLS problem.

Impact: You waste time troubleshooting TLS when the real issue is just wrong protocol.

Resolution:

```
# SSH to SDDC Manager
ssh vcf@sddc-manager.lab.local

# WRONG — misleading "wrong version number" error:
curl -sk https://localhost:7200/health
# Error: curl: (35) error:1408F10B:SSL routines:ssl3_get_record:wrong version
number

# CORRECT — use HTTP:
curl -s http://localhost:7200/health
# Returns: {"status":"UP"}

# All internal service ports and their protocols:
# Port 7200 — domainmanager — HTTP  (not HTTPS!)
# Port 7300 — operationsmanager — HTTP
# Port 7400 — lcm — HTTP
# Port 443  — Nginx reverse proxy — HTTPS (this is what external clients use)

# Quick check for all services:
for port in 7200 7300 7400; do
  STATUS=$(curl -s --connect-timeout 3 http://localhost:$port/health 2>/dev/null)
  echo "Port $port: $STATUS"
done
```

Issue #26: NSX Adapter Credential Fields Must Use Uppercase Names

Problem: When creating NSX adapter credentials via Suite-API, the field names must be `USERNAME` and `PASSWORD` (all uppercase). Using `USER` or `user` fails with "USERNAME is mandatory".

Impact: No documentation specifies the exact field names. Trial and error is the only way to discover this.

Resolution — Complete credential and adapter creation:

```
# bash on workstation — Suite-API REST flow ($TOKEN from prior step)
# STEP 1: Create the credential with CORRECT field names
CRED_RESP=$(curl -sk -X POST "https://192.168.1.77/suite-api/api/credentials" \
  -H "Authorization: vRealizeOpsToken $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "name": "nsx-vip.lab.local",
    "adapterKindKey": "NSXTAdapter",
    "credentialKindKey": "NSXTCREDENTIAL",
    "fields": [
      {"name": "USERNAME", "value": "admin"},
      {"name": "PASSWORD", "value": "<your-password>"}
    ]
  }')

echo "$CRED_RESP" | python3 -m json.tool
CRED_ID=$(echo "$CRED_RESP" | python3 -c "import
sys,json;print(json.load(sys.stdin)['id'])")
echo "Credential ID: $CRED_ID"

# WRONG field names that will fail:
# {"name": "USER", "value": "admin"}        → "USERNAME is mandatory"
# {"name": "user", "value": "admin"}        → "USERNAME is mandatory"
# {"name": "username", "value": "admin"}    → "USERNAME is mandatory"
# {"name": "PASS", "value": "..."}          → "PASSWORD is mandatory"

# STEP 2: Create the NSX adapter using the credential
curl -sk -X POST "https://192.168.1.77/suite-api/api/adapters" \
  -H "Authorization: vRealizeOpsToken $TOKEN" \
  -H "Content-Type: application/json" \
  -d "{
    \"name\": \"nsx-vip.lab.local\",
    \"description\": \"NSX Manager\",
    \"adapterKindKey\": \"NSXTAdapter\",
    \"resourceIdentifiers\": [
      {\"name\": \"NSXTHOST\", \"value\": \"nsx-vip.lab.local\"},
      {\"name\": \"AUTO_DISCOVERY\", \"value\": \"true\"},
```

```
      {\"name\": \"ENABLE_ALERTS_FROM_NSX\", \"value\": \"false\"},
      {\"name\": \"VCURL\", \"value\": \"vcenter.lab.local\"},
      {\"name\": \"VMEntityVCID\", \"value\": \"92109cf0-ad3b-4ffa-8972-
a77bb7fadacf\"},
      {\"name\": \"NSX_CLUSTER_ID\", \"value\": \"6c55d856-ab96-4190-8495-
3cc8cb23450c\"}
    ],
    \"credential\": {\"id\": \"$CRED_ID\"},
    \"collectorId\": 2
  }"
```

PowerShell equivalent:

```
# PowerShell 5.1+ on Windows workstation ($TOKEN, $hdr from prior step)
[Net.ServicePointManager]::ServerCertificateValidationCallback = { $true }
$hdr = @{ Authorization = "vRealizeOpsToken $TOKEN"; Accept = 'application/json'
}

# STEP 1: Create the credential with CORRECT field names (USERNAME / PASSWORD
uppercase)
$credBody = @{
    name              = 'nsx-vip.lab.local'
    adapterKindKey    = 'NSXTAdapter'
    credentialKindKey = 'NSXTCREDENTIAL'
    fields            = @(
        @{ name = 'USERNAME'; value = 'admin' },
        @{ name = 'PASSWORD'; value = '<your-password>' }
    )
} | ConvertTo-Json -Depth 5

$CRED_RESP = Invoke-RestMethod -Method Post `
    -Uri 'https://192.168.1.77/suite-api/api/credentials' `
    -Headers $hdr -ContentType 'application/json' -Body $credBody
$CRED_RESP | ConvertTo-Json -Depth 5
$CRED_ID = $CRED_RESP.id
"Credential ID: $CRED_ID"

# WRONG field names that will fail:
#   USER, user, username   → "USERNAME is mandatory"
#   PASS                   → "PASSWORD is mandatory"

# STEP 2: Create the NSX adapter using the credential
$adapterBody = @{
```

```
    name                = 'nsx-vip.lab.local'
    description         = 'NSX Manager'
    adapterKindKey      = 'NSXTAdapter'
    resourceIdentifiers = @(
        @{ name = 'NSXTHOST';               value = 'nsx-vip.lab.local' },
        @{ name = 'AUTO_DISCOVERY';         value = 'true' },
        @{ name = 'ENABLE_ALERTS_FROM_NSX'; value = 'false' },
        @{ name = 'VCURL';                  value = 'vcenter.lab.local' },
        @{ name = 'VMEntityVCID';           value = '92109cf0-ad3b-4ffa-8972-
a77bb7fadacf' },
        @{ name = 'NSX_CLUSTER_ID';         value = '6c55d856-ab96-4190-8495-
3cc8cb23450c' }
    )
    credential          = @{ id = $CRED_ID }
    collectorId         = 2
} | ConvertTo-Json -Depth 6

Invoke-RestMethod -Method Post `
    -Uri 'https://192.168.1.77/suite-api/api/adapters' `
    -Headers $hdr -ContentType 'application/json' -Body $adapterBody
```

Issue #27: Gemfire Cache Takes 5-10 Minutes to Populate After Cluster Init

Problem: After VCF Operations cluster initialization, the Gemfire distributed cache takes 5-10 minutes to fully populate. Roles, users, adapters, and other data may not appear in API responses during this window.

Impact: Administrators conclude data is missing and take unnecessary corrective action (like trying to recreate roles or reinitialize the cluster).

Resolution — Wait and verify:

```
# bash on workstation — Suite-API polling loop
# After cluster initialization, run this monitoring loop:
TOKEN=$(curl -sk -X POST https://192.168.1.77/suite-api/api/auth/token/acquire \
  -H "Content-Type: application/json" \
  -d '{"username":"admin","password":"<your-password>","authSource":"local"}' \
  | python3 -c "import sys,json;print(json.load(sys.stdin)['token'])")

for i in $(seq 1 20); do
  ROLES=$(curl -sk -H "Authorization: vRealizeOpsToken $TOKEN" \
    https://192.168.1.77/suite-api/api/auth/roles 2>/dev/null \
    | python3 -c "import sys,json;d=json.load(sys.stdin);print(len(d.get('roles',
[])))" 2>/dev/null || echo "0")
```

```
  ADAPTERS=$(curl -sk -H "Authorization: vRealizeOpsToken $TOKEN" \
    https://192.168.1.77/suite-api/api/adapters 2>/dev/null \
    | python3 -c "import
sys,json;d=json.load(sys.stdin);print(len(d.get('adapterInstancesInfoDto',[])))"
2>/dev/null || echo "0")
  echo "[$(date +%H:%M:%S)] Roles: $ROLES | Adapters: $ADAPTERS"
  if [ "$ROLES" -gt 0 ] && [ "$ADAPTERS" -gt 0 ] 2>/dev/null; then
    echo "Gemfire cache populated — system ready."
    break
  fi
  sleep 30
done
```

PowerShell equivalent:

```
# PowerShell 5.1+ on Windows workstation
[Net.ServicePointManager]::ServerCertificateValidationCallback = { $true }

$body = @{
    username = 'admin'; password = '<your-password>'; authSource = 'local'
} | ConvertTo-Json
$TOKEN = (Invoke-RestMethod -Method Post `
    -Uri 'https://192.168.1.77/suite-api/api/auth/token/acquire' `
    -ContentType 'application/json' -Body $body).token
$hdr = @{ Authorization = "vRealizeOpsToken $TOKEN"; Accept = 'application/json'
}

for ($i = 1; $i -le 20; $i++) {
    $ROLES = 0; $ADAPTERS = 0
    try {
        $r = Invoke-RestMethod -Uri 'https://192.168.1.77/suite-
api/api/auth/roles' `
            -Headers $hdr -TimeoutSec 10
        if ($r.roles) { $ROLES = $r.roles.Count }
    } catch {}
    try {
        $a = Invoke-RestMethod -Uri 'https://192.168.1.77/suite-api/api/adapters'
`
            -Headers $hdr -TimeoutSec 10
        if ($a.adapterInstancesInfoDto) { $ADAPTERS =
$a.adapterInstancesInfoDto.Count }
    } catch {}
```

```
    "[{0}] Roles: {1} | Adapters: {2}" -f (Get-Date -Format HH:mm:ss), $ROLES,
$ADAPTERS
    if ($ROLES -gt 0 -and $ADAPTERS -gt 0) {
        'Gemfire cache populated — system ready.'
        break
    }
    Start-Sleep -Seconds 30
}
```

Expected timeline after cluster init:

Time	Roles Visible	Adapters Visible
0-2 min	0	0
2-5 min	0-3	0-5
5-10 min	All (e.g., 5)	All (e.g., 15+)

Issue #28: VCF Operations HSQLDB Reset Required After Unclean Shutdown

Problem: An unclean shutdown of VCF Operations leaves the HSQLDB and Gemfire cache in an inconsistent state, causing `INITIALIZATION_FAILED`. There is no automatic recovery mechanism.

Impact: VCF Operations is completely non-functional until manual HSQLDB reset.

Resolution — Complete HSQLDB reset procedure:

```
# ============================================================
# VCF OPERATIONS HSQLDB RESET — COMPLETE PROCEDURE
# ============================================================

# STEP 1: SSH to VCF Operations node as root
ssh root@192.168.1.77
# Password: <your-password>

# STEP 2: Verify the problem — check cluster state
curl -sk https://localhost/casa/cluster/status
# Expected: "state": "INITIALIZATION_FAILED"

# STEP 3: Stop all VCF Operations services
systemctl stop vmware-casa
systemctl stop vmware-vcops-watchdog
```

```
# STEP 4: Backup the HSQLDB script file
cp /storage/db/casa/webapp/hsqldb/casa.db.script \
   /storage/db/casa/webapp/hsqldb/casa.db.script.bak

# STEP 5: Edit the HSQLDB — change initialization state
# Find the line containing "initialization_state":"FAILED"
grep -n "initialization_state" /storage/db/casa/webapp/hsqldb/casa.db.script
# Note the line number

# Option A: Use sed to do the replacement
sed -i 's/"initialization_state":"FAILED"/"initialization_state":"NONE"/g' \
  /storage/db/casa/webapp/hsqldb/casa.db.script

# Option B: Use vi if you prefer manual editing
# vi /storage/db/casa/webapp/hsqldb/casa.db.script
# Find: "initialization_state":"FAILED"
# Replace with: "initialization_state":"NONE"
# Save and exit (:wq)

# Verify the change was made
grep "initialization_state" /storage/db/casa/webapp/hsqldb/casa.db.script
# Should show: "initialization_state":"NONE"

# STEP 6: Clear the HSQLDB log file (forces clean state)
> /storage/db/casa/webapp/hsqldb/casa.db.log

# STEP 7: Clear admin password hash (forces regeneration)
cat > /storage/vcops/user/conf/adminuser.properties << 'EOF'
#Properties for vCOps user 'admin'
username=admin
hashed_password=
EOF

# STEP 8: Get the SHA1 thumbprint (used during initialization)
THUMBPRINT=$(openssl x509 -in /storage/vcops/user/conf/ssl/cert.pem \
  -noout -fingerprint -sha1 | sed 's/SHA1 Fingerprint=//')
echo "SHA1 Thumbprint: $THUMBPRINT"

# STEP 9: Restart services
systemctl start vmware-casa
systemctl start vmware-vcops-watchdog

# STEP 10: Wait for CASA to fully start (monitor logs)
tail -f /usr/lib/vmware-casa/casa-webapp/logs/casa.log | grep -i
'startup\|init\|error'
```

```
# Wait until you see "Started Application" or similar startup message
# Press Ctrl+C to stop tailing

# STEP 11: Trigger cluster initialization
curl -sk -X POST https://localhost/casa/cluster/init \
  -H "Content-Type: application/json"

# STEP 12: Verify cluster status
curl -sk https://localhost/casa/cluster/status
# Expected: "cluster_state": "INITIALIZED"

# STEP 13: Verify slice is online
curl -sk https://localhost/casa/sysadmin/slice/online_state
# Expected: "onlineState":"ONLINE"

# STEP 14: Wait 5-10 minutes for Gemfire cache to populate (see Issue #27)
# Then log in to the VCF Operations UI at https://192.168.1.77/
# Username: admin
# Password: <your-password>
```

---

8. Photon OS 5 Networking & OVFTool

Issue #29: OVFTool Silently Drops vApp Properties with InstanceId

Problem: When deploying a VCF Operations OVA via OVFTool, properties that have an `InstanceId` in the OVF descriptor (e.g., `VMware_Aria_Operations`) are silently ignored when specified with `--prop:key=value`. OVFTool reports "Completed successfully" but the properties are not set.

Impact: The VM deploys successfully but boots with no network configuration — all IPv4 properties (address, gateway, netmask, type) and networking properties (DNS, domain, searchpath) are blank. The appliance falls back to DHCP.

Symptoms:

- OVFTool output shows warnings like: `OVF property with key: 'ipv4_address' does not exists.`
- The deployment completes with exit code 0 (success)
- The VM boots with DHCP or no IP

How to identify affected properties — probe the OVA:

```
# bash on workstation — ovftool is cross-platform
ovftool "Operations-Appliance-9.0.1.0.24960351.ova" 2>&1 | grep -A1 "InstanceId"
```

PowerShell equivalent:

```
# PowerShell 5.1+ on Windows workstation
# ovftool ships with VMware tooling on Windows; call it directly and filter with Select-String.
# Use call operator if path contains spaces, e.g. & 'C:\Program Files\VMware\VMware OVF Tool\ovftool.exe'
$ova = 'Operations-Appliance-9.0.1.0.24960351.ova'
& ovftool $ova 2>&1 | Select-String -Pattern 'InstanceId' -Context 0,1
```

Properties with `InstanceId` output like:

```
  Key:         ipv4_address
  InstanceId   VMware_Aria_Operations
```

These properties cannot be set with simple `--prop:ipv4_address=value` syntax.

Resolution — Set vApp properties in vCenter after deployment:

```
1. Deploy the OVA with OVFTool (network properties will be blank)
2. Do NOT power on the VM
3. In vCenter, navigate to the VM → Configure → vApp Options → Properties
4. Set all network properties via SET VALUE (see the Photon OS 5 Desired State procedure (Broadcom KB 425535))
5. Power on the VM
```

Alternative — Deploy via vSphere Client:

The vSphere Client OVF deployment wizard correctly maps InstanceId properties through the "Customize template" step. If you need the properties set during deployment, use the GUI wizard instead of OVFTool.

Issue #30: VCF Operations `nmctl` Not Available (Fleet Manager Only)

Problem: Broadcom KB articles recommend using `nmctl set-static` to configure static IP on VCF appliances running Photon OS 5. However, `nmctl` (network-config-manager) is only installed on the **Fleet Manager** appliance

— it is not available on the **VCF Operations** appliance.

Impact: Following Broadcom's documented procedure on the VCF Operations appliance fails with `command not found`. Engineers waste time trying to find the correct tool.

Symptoms:

```
root@vcf-ops [ ~ ]# nmctl set-static dev eth0 a 192.168.1.77/24 gw 192.168.1.1
-bash: nmctl: command not found
```

Resolution — Use vApp properties instead:

The VCF Operations appliance does not use `nmctl` for network configuration. The correct approach is:

1. Set vApp properties in vCenter (see the Photon OS 5 Desired State procedure (Broadcom KB 425535)) — this is the permanent fix
2. For temporary connectivity, use `ip addr add` from the VM console:

```
ip addr add 192.168.1.77/24 dev eth0
ip route add default via 192.168.1.1 dev eth0
```

Note: `ip addr add` changes are lost on reboot. Only vApp properties persist.

---

9. Aria Suite / LCM / VCF Automation

Issues discovered during VCF Automation (VMSP/VCFA) deployment work in April 2026.

Issue #31: VMware API Returns Memory in Different Units

Discovered: April 14, 2026 **Component:** vCenter PropertyCollector API **Severity:** Critical — causes incorrect health monitoring

The vCenter `ClusterComputeResourceSummary` API returns two memory properties in **different units**:

- `summary.totalMemory` — returns **bytes**
- `summary.effectiveMemory` — returns **megabytes**

This is not documented in a way that makes the difference obvious. If you calculate memory usage as `(totalMemory - effectiveMemory) / totalMemory * 100`, you get ~99% because you're

subtracting megabytes from bytes.

Fix: Multiply `effectiveMemory` by `1048576` (1024 * 1024) to convert MB to bytes before any calculation.

```
tm = int(summary.totalMemory)        # bytes
em = int(summary.effectiveMemory) * 1048576  # MB -> bytes
mem_percent = ((tm - em) / tm) * 100
```

Issue #32: NSX HTTP Service Doesn't Auto-Start After Reboot

Discovered: April 14-15, 2026 **Component:** NSX Manager 9.0.1 **Severity:** High — NSX API completely unreachable

After a cold start or reboot, the NSX Manager VM will respond to ping (ICMP) but the API on port 443 is unreachable. All other NSX services start correctly, but the `http` reverse proxy service remains `stopped`.

Symptoms:

- VM pings but `curl https://nsx-vip:443` times out
- `get services` shows `http` service state: `stopped`
- All other services show `running`

Fix: From the NSX CLI:

```
start service http
```

Prevention: After every reboot, SSH to NSX Manager and run `get services` to verify `http` is running. This is not the same as the NSX Manager VM being up — ping is not sufficient.

Issue #33: LCM UI Creates Orphaned UI_INPROGRESS Database Records

Discovered: April 15-16, 2026 **Component:** VCF Operations Fleet Management / Aria Suite Lifecycle Manager **Severity:** Critical — blocks all future deployments

The VCF Operations Fleet Management UI creates `UI_INPROGRESS` records in the LCM PostgreSQL database (`vm_lcops_environments` table) every time you open a product deployment page (automation, identity broker, operations-networks). If you navigate away without submitting, close the browser, or the session times out, these records are **never cleaned up**.

These orphaned records trigger the error: "*Parallel Deployments of automation and identity broker are not allowed.*"

Affected tables (in dependency delete order):

1. `vm_lcops_node_properties`
2. `vm_lcops_product_nodes`
3. `vm_lcops_product_properties`
4. `vm_lcops_product`
5. `vm_lcops_infrastructure_properties`
6. `vm_lcops_environments`
7. `vm_rs_request` (independent)
8. `vm_engine_execution_request` (independent)

Fix: Stop `vrlcm-server`, delete from all tables in order (deepest child first), restart. Must also restart `vmware-vcops-web` on VCF Operations appliance to clear cached state.

References: Broadcom KB 388305, GitHub Holodeck Issue #91

Issue #34: VCF Automation Creates Both `vra` AND `vmsp` Product Entries

Discovered: April 16, 2026 **Component:** Aria Suite Lifecycle Manager **Severity:** Critical — hidden entries block cleanup

When deploying VCF Automation, the LCM engine creates entries under **two different product IDs**: `vra` (the VCF Automation product) and `vmsp` (the VM Service Platform). If you only clean `vra` entries during database cleanup, the hidden `vmsp` entries remain and continue blocking deployments.

Symptoms: After cleaning `vm_lcops_product WHERE id='vra'`, running `SELECT id FROM vm_lcops_product;` still shows `vmsp` entries.

Fix: Always clean ALL product IDs that aren't `vrops` or `vrli`:

```
DELETE FROM vm_lcops_product WHERE id NOT IN ('vrops','vrli');
```

Issue #35: Parallel Deployment Check Runs in VCF Operations Frontend, Not LCM

Discovered: April 16, 2026 **Component:** VCF Operations Web Application **Severity:** Critical — cleaning LCM database alone doesn't fix it

The "Parallel Deployments of automation and identity broker are not allowed" check is performed by the **VCF Operations web application** (vmware-vcops-web service on the VCF Ops appliance), not by the LCM engine. VCF Operations caches the deployment state in its application memory. Cleaning the LCM database alone does not clear this cache.

Fix: Must restart `vmware-vcops-web` on the VCF Operations appliance (192.168.1.77) in addition to cleaning the LCM database:

```
# On VCF Operations appliance
systemctl restart vmware-vcops-web
```

Workaround: Bypass the UI entirely by submitting deployment requests via the LCM REST API (`POST /lcm/lcops/api/v2/environments`), which does not have this frontend check.

Issue #36: VCF Automation ipPool Must Be in nodes[0].properties

Discovered: April 16, 2026 **Component:** LCM REST API / VMSP Bootstrap **Severity:** Critical — deployment fails with LCMVMSP10000

When deploying VCF Automation via the LCM API, the `ipPool` , `primaryVip` , and `internalClusterCidr` properties must be placed in `products[0].nodes[0].properties` , **not** in `products[0].properties` . The VMSP BootstrapVMSPTask reads these from the node properties, not the product properties.

Placing them in the wrong location causes: `LCMVMSP10000: Certain Mandatory properties are missing in the component spec properties : [ipPool]`

Correct structure:

```
"nodes": [{
  "type": "vcfa-primary",
  "properties": {
    "vmNamePrefix": "vcfa-mgmt",
    "primaryVip": "192.168.1.91",
    "ipPool": "192.168.1.92-192.168.1.94",
    "internalClusterCidr": "198.18.0.0/15"
  }
}]
```

How to get the correct format: Use the UI's **Export Configuration** button on the Summary page to capture the exact payload the UI sends.

Issue #37: Suite-API Auth Header Changed from vRealizeOpsToken to OpsToken

Discovered: April 15, 2026 **Component:** VCF Operations Suite API **Severity:** High — old scripts will fail

The VCF Operations Suite API authorization header has been updated from the legacy format to a new format:

- **Deprecated:** `Authorization: vRealizeOpsToken <token>`
- **Current:** `Authorization: OpsToken <token>`

The old format still works for backward compatibility but is deprecated and may be removed in a future release.

Fix: Update all scripts and automation to use `OpsToken`:

```
# bash on workstation
curl -sk -H "Authorization: OpsToken $TOKEN" https://vcf-ops/suite-api/api/...
```

PowerShell equivalent:

```
# PowerShell 5.1+ on Windows workstation
[Net.ServicePointManager]::ServerCertificateValidationCallback = { $true }
$hdr = @{ Authorization = "OpsToken $TOKEN"; Accept = 'application/json' }
Invoke-RestMethod -Uri 'https://vcf-ops/suite-api/api/...' -Headers $hdr
```

---

10. vSAN & DVS Edge Cases (May 2026)

Issues discovered during esxi02/esxi03 vSAN disk-group rebuild work, May 9–11, 2026.

Issue #38: vSAN OSA capacity disk on nested ESXi — `.vmx virtualSSD` flag required, NOT SATP rule

Discovered: May 9, 2026 **Component:** vSAN OSA on nested ESXi (VMware Workstation) **Severity:** High — perf degradation (70 ms write latency, 0% LCC hit rate)

Broadcom KB 341618 documents using `esxcli storage nmp satp rule add --satp=VMW_SATP_LOCAL --device=<dev> --option "enable_local enable_ssd"` followed by host reboot to flip `Is SSD: false` to `Is SSD: true` on devices not auto-detected as SSD. **This path does NOT work for nested ESXi running on VMware Workstation.** Verified empirically on esxi02 2026-05-09: SATP rule applied, host rebooted, `Is SSD: false` persisted.

Why SATP fails on nested: SATP rules tag at the VMkernel layer. For nested ESXi, `Is SSD` is propagated FROM the outer hypervisor (VMware Workstation) TO the guest via `sataX:Y.virtualSSD = "1"` in the outer `.vmx`. No amount of in-guest SATP manipulation can override the device-type metadata coming from the outer Workstation VM. **Layer 1 (outer `.vmx`) wins.**

Symptoms:

- `esxcli storage core device list -d <device>` reports `Is SSD: false` even after the SATP rule path
- vSAN disk group runs in hybrid mode (cache + HDD-style capacity) — 70+ ms write latency, 0% LCC hit rate
- Broadcom KB 341618 procedure completes without error but has no effect

Fix (verified on esxi02 2026-05-09, esxi03 2026-05-10):

1. Power off the nested ESXi VM from VMware Workstation (host must be in MM with vSAN evacuated first)
2. Open the outer `.vmx` (e.g., `Clone of esxi02.lab.local.vmx`) in a text editor
3. Locate the existing block of `sataX:Y.virtualSSD = "1"` lines (typically `sata0:0` and `sata0:2`)
4. Add lines for each capacity slot that is missing the flag:

   ```
   sata0:3.virtualSSD = "1"
   sata0:4.virtualSSD = "1"
   ```

5. Save the file (UTF-8, no BOM)
6. Power on the nested ESXi VM from VMware Workstation
7. SSH and verify:

   ```
   esxcli storage core device list -d
   t10.ATA_____VMware_Virtual_SATA_Hard_Drive__________04000000000000000001 |
   grep "Is SSD"
   ```

   Should now return `Is SSD: true`

Bonus finding: vSAN AUTO-SETS `IsCapacityFlash: 1` on the affected capacity disks once `Is SSD: true` is reported at boot — the explicit `esxcli vsan storage tag add -t capacityFlash` step from older procedures is no longer strictly required, though the disk group itself must still be removed and re-created to flip from hybrid to all-flash internal mode.

Cross-reference: Full procedure in `vSAN-OSA-Disk-Group-Rebuild-Handbook-2026-05-08.md` §4–6.

Audit notes: Searched Broadcom KB 2026-05-11. KB 341618 covers only physical hosts (no mention of nested ESXi); VMware KB 1006827 covers `virtualSSD` config for Workstation/Fusion but does not describe its interaction with nested ESXi vSAN SATP. Gap confirmed.

Issue #39: `esxcli vsan debug object health summary get` `data-move` count EXCLUDES "Decommissioning" intent

Discovered: May 10, 2026 **Component:** vSAN OSA / vSAN ESA observability **Severity:** Medium — false "evacuation complete" signal during MM

The `esxcli vsan debug object health summary get` command reports a `data-move` count that is widely used to detect when vSAN evacuation has finished during Maintenance Mode-with-Full-Data-Migration. **The `data-move` counter does NOT include components in `Decommissioning` intent state** — which is the FINAL step of MM evacuation (when components are being moved off the host that's entering MM). The result: scripts and dashboards report "evacuation complete" while up to tens of GB of decommissioning still runs in the background.

Symptoms:

- `esxcli vsan debug object health summary get` shows `data-move: 0`, `healthy: 162` (all objects accounted for as healthy)
- BUT `esxcli vsan debug resync summary get` shows `Total Bytes Left To Resync: 45+ GB`, `Total Number Of Resyncing Objects: 4+`
- `esxcli vsan debug resync list` shows entries with `Intent: Decommissioning`
- Maintenance Mode does NOT complete despite the "all clear" health summary

Same gap also exists in PowerCLI `Get-VsanResyncingComponent` — it misses Decommissioning intent the same way.

Fix — use the bytes-count probe as the true completion signal:

```
# AUTHORITATIVE check (catches Decommissioning):
esxcli vsan debug resync summary get
# Wait until:
#   Total Number Of Resyncing Objects: 0
#   Total Bytes Left To Resync: 0
#   Total GB Left To Resync: 0.00
```

In automation, exit-criterion should require BOTH:

- `data-move == 0` (from health summary)
- `Total GB Left To Resync == 0.00` (from resync summary)

If only checking `data-move`, the watcher fires `RESYNC COMPLETE` prematurely while ~5–10% of evacuation work is still in flight. Verified on esxi03 evacuation 2026-05-10 — watcher fired `data-move=0` with 46 GB and 4 objects still in `Decommissioning` intent.

Audit notes: Searched Broadcom KB 2026-05-11. Generic `esxcli vsan debug` documentation exists; vSAN community thread "vSAN Resync - Intent Decommissioning" on community.broadcom.com discusses Decommissioning intent in general but no Broadcom KB documents that `data-move` health-summary counter excludes it.

Issue #40: `Hostsvc.DistEsxClusterStoreManager` `clusterAdmin runReplica` exit:2 blocks MM completion

Discovered: May 10, 2026 **Component:** vSphere Distributed Service Engine Cluster Store (etcd on port 2379) **Severity:** High — host stuck in "Entering Maintenance Mode" state indefinitely

vSphere 9.0 introduces a Cluster Store (distributed etcd-based config service on port 2379, one replica per cluster host). When a host enters MM, vSphere must migrate the host's Cluster Store replica to another host. If the `clusterAdmin runReplica` command fails (exit code 2, often with empty stderr), MM transition **never completes** even after vSAN evacuation is fully done.

Symptoms:

- vCenter UI shows MM task at 95–99% indefinitely, then fails with "general vSAN error" or generic MM failure
- `/var/log/hostd.log` on the source host shows repeating entries:

```
Hostsvc.DistEsxClusterStoreManager Admin command failed;
  '/usr/lib/vmware/clusterAgent/bin/clusterAdmin ++group=hostd-tmp,mem=30
   cluster runReplica <cluster-id>:domain-c10 <target>.lab.local:2379
   <source>.lab.local --hostname <source>.lab.local --opid <opid>',
  exit: 2, stderr:
```

- vSAN evacuation completes (`data-move=0`, `GB-left=0`) but host `esxcli system maintenanceMode get` still reports `Disabled`

Root cause indicators: etcd port 2379 connectivity between source and target host, certificate/auth state of cluster store agents, or cluster store agent not running on target.

Workaround — manual MM after verifying evacuation:

After confirming vSAN evacuation is truly complete (per Issue #39 — BOTH health-summary `data-move=0` AND resync-summary `GB-Left=0.00`):

```
# Verify on the host being placed in MM:
esxcli vsan debug object health summary get | grep -E 'inaccessible|data-move|healthy'
esxcli vsan debug resync summary get   # Total GB Left = 0.00

# Force MM directly at the host level (bypassing vCenter coordination):
esxcli system maintenanceMode set --enable true

# Verify:
esxcli system maintenanceMode get
# Expected: Enabled
```

vCenter will eventually sync up its MM state from the host. Continue with disk-group rebuild as normal.

Caveats:

- Only use this bypass after verifying vSAN evacuation is truly complete — otherwise data on source host may go inaccessible
- vCenter UI may continue to show the MM task as failed; cancel it after manually entering MM
- After exiting MM normally, vCenter resync will pull the host back into normal management

Audit notes: Searched Broadcom KB 2026-05-11. Generic MM failure KBs exist (KB 434515, 374379, 416094) covering DRS rules, vMotion failures, NSX scenarios — but no Broadcom article identifies `DistEsxClusterStoreManager` / `clusterAdmin runReplica` exit:2 as a specific failure mode or documents the manual MM bypass for that case. Gap confirmed.

---

12. NSX 9.0 vDS Replacement / vLCM Lifecycle Edge Cases (May 2026)

Issue #43: NSX `?action=clear` on `nsx-ownership` fails with error_code 8012 when vCenter no longer has the referenced vDS UUID

Problem: After a vDS is replaced in a VCF-managed cluster (old vDS removed, new vDS created with a different UUID), NSX's ownership table still records the original vDS UUID as "owned by NSX" on the Compute Manager. Attempts to clear that ownership via the documented API path fail because NSX's `?action=clear` makes a callback to vCenter to release the resource, vCenter responds "no such object", and NSX returns `error_code 8012`.

Impact:

- vCenter alarm "The vSphere Distributed Switch corresponding to the proxy switches `<orphan-uuid>` does not exist" cannot be resolved through any NSX-UI or NSX-API path

- Any NSX cleanup workflow that touches the ownership table (TNP/TNC detach, host re-prep) fails before it starts
- VCF Operations / SDDC Manager UI workflows may also surface this error to operators with no documented recovery

Symptoms:

```
PUT /api/v1/managed-objects/switching/nsx-ownership/<cm-id>?action=clear
{"mo_id":"<orphan-vds-uuid>"}
```

Response:

```
{
  "details": "The requested object : <orphan-vds-uuid> could not be found. Object identifiers are case sensitive.",
  "httpStatus": "BAD_REQUEST",
  "error_code": 8012,
  "module_name": "NsxSwitching service",
  "error_message": "Unable to clear NSX ownership from MO <cm-id>/<orphan-vds-uuid>."
}
```

Workarounds attempted, ALL REJECTED:

Attempt	NSX Response
`?action=clear&force=true`	`error_code 268` — "Request has unknown parameters: force."
`?action=force_clear`	`error_code 255` — "value force_clear of property action is not one of the allowed values [set, clear]"
`?action=clear_local`	`error_code 255` — same enum-validation failure
`DELETE /api/v1/managed-objects/switching/nsx-ownership/<cm>`	HTTP 405 Method Not Allowed
`?action=set` with live vDS UUID, then re-try `?action=clear` with orphan	`set` returns HTTP 200 but does not displace the orphan entry; subsequent clear still 8012

Workaround that DOES work (verified 2026-05-16):

There is no API-level recovery. The supported path appears to be a full NSX un-prep + re-prep cycle on the cluster, which is itself blocked (see Issue #45 below). Currently the only viable option is:

1. Open a Broadcom SR (template: `Broadcom-SR-NSX-Orphan-vDS-Cleanup-2026-05-16.md`)
2. Accept the residual NSX ownership record; it is internal NSX state with no production-traffic impact
3. The vCenter "proxy switches" alarm clears separately when the TNP-level `host_switch_id` is updated (PUT on `/policy/api/v1/infra/host-transport-node-profiles/<tnp-id>` works regardless of the ownership-table state)

Audit note: Searched Broadcom KB 2026-05-16 for `error_code 8012`, `nsx-ownership clear`, `Unable to clear NSX ownership from MO`. No KB matches. Related but non-applicable: KB 387942 (orphan host entries), KB 387148 (orphan portgroups), KB 379010 (DVS out-of-sync), KB 370575 (NVDS→VDS stale switch), KB 298537 (orphan NSX-T objects). None covers post-vDS-replacement ownership-table state.

Discovered: 2026-05-15 during NSX trust-and-CM repair work. Re-verified 2026-05-16 after migrating all VM references off the orphan vDS — the ownership clear still fails because the orphan UUID never existed in vCenter at all (or was deleted long ago); the ownership record is pure NSX-internal state.

Issue #44: NSX per-host Transport Node PUT to change `host_switch_id` fails with error_code 9560 (single-transaction TZ relocation guard)

Problem: Per-host Transport Node records (under `/policy/api/v1/infra/sites/default/enforcement-points/default/host-transport-nodes/<tn-id>`) hold their own copy of `host_switch_id` in `host_switch_spec`. When the cluster's vDS UUID has changed and the TNP has been corrected at the parent level, the per-host TNs still carry the orphan UUID and are not auto-reconciled. Attempts to PUT a corrected `host_switch_id` on a TN are rejected by an NSX safety guard that refuses to relocate a Transport Zone binding between two HostSwitches in a single transaction.

Impact:

- Per-host TN state remains pinned to the orphan vDS UUID even after TNP and TNC are correct
- NSX UI shows the alarm on a per-host basis: "*Transport Zone TransportZone/* `<tz-id>` *cannot be relocated from HostSwitch* `<orphan>` *to HostSwitch* `<live>` *in a single transaction. Delete HostSwitch from current Transport Zone to add later to another Transport Zone.*"
- TEP networking and overlay traffic continue to work (because the host's actual port-level config is fine), but the realization state is inconsistent

Symptoms:

```
PUT /policy/api/v1/infra/sites/default/enforcement-points/default/host-transport-nodes/<tn-id>
{... host_switch_spec.host_switches[0].host_switch_id changed from orphan to live UUID ...}
```

Response:


```
{
  "httpStatus": "BAD_REQUEST",
  "error_code": 9560,
  "module_name": "NsxSwitching service",
  "error_message": "Transport Zone TransportZone/<tz-realized-id> cannot be
relocated from HostSwitch <orphan-uuid> to HostSwitch <live-uuid> in a single
transaction. Delete HostSwitch from current Transport Zone to add later to another
Transport Zone."
}
```


Why the guard fires even after fixing the TNP: The TZ realized-binding (`TransportZone/<realized-id>`) is independent of the TNP. It was created when the host was first prepped; it points at the host_switch UUID that existed at that time. Updating the TNP does not migrate existing TZ bindings.

Workaround attempts:

Attempt	Result
Two-PUT pattern: first PUT remove `transport_zone_endpoints`, second PUT add to new host_switch_id	Not yet attempted in production (high risk of leaving host unprep'd between PUTs)
TNC DELETE to force re-prep	Blocked by Issue #45
Migrate all VMs off old vDS first	Does not help — TZ binding is at host-level, independent of VMs

Workaround that DOES work: None API-level confirmed for this scenario. Best path is the full TNC detach + reattach via the supported VCF Operations / SDDC Manager UI workflow (un-prep + re-prep ~30 min per host).

Audit note: Searched Broadcom KB 2026-05-16 for `error_code 9560` , `Transport Zone cannot be relocated` , `HostSwitch single transaction` . No KB matches. The guard message itself suggests a two-step workaround exists (`Delete HostSwitch from current Transport Zone to add later to another Transport Zone`) — but **no Broadcom KB or TechDoc documents the API calls for the two-step procedure**.

Discovered: 2026-05-15 during NSX TNP repair work — Phase 3 of the NSX trust+CM+TNP plan. Re-verified 2026-05-16 (post VM migration); still 9560.

Issue #46: VCF Operations 9.0.1 "Import Certificate" UI has no Private Key field — expects a Fleet-side CSR workflow not surfaced in the dialog

Problem: When replacing the Fleet (vRSLCM) cert via VCF Operations Admin UI → Certificates → VCF Management → Fleet row → Replace Certificate, the **Import Certificate** dialog asks only for:

- Name
- Source (Paste Text)
- Server Certificate (PEM)
- Certificate Authority (PEM)

There is **no Private Key field.** The dialog silently assumes the operator pre-generated a Certificate Signing Request through Fleet itself (which would keep the private key internally on Fleet), signed the CSR externally, and is now pasting the signed cert back.

Operators following the VCFA Airtight Plan (which says "generate a fresh keypair on Fleet, import via UI") fall into a trap: pasting an externally-generated cert without a matching Fleet-internal CSR results in a cert/key mismatch that surfaces as a generic banner: "*Certificate replacement for appliance lcm.lab.local has failed. Failed to perform specified operation. Certificate replace operation for VCF Operations Fleet Management failed.*"

The dialog gives no specific reason and offers no path to upload the matching private key.

Impact:

- The KB-411354-prescribed Fleet cert replace workflow fails for any operator who generated the keypair outside Fleet
- Wasted ~30+ minutes per operator who didn't already know about the missing field
- VCFA deploy preconditions block until the cert is correctly installed

Symptoms:

- Red banner: "Certificate replacement for appliance lcm.lab.local has failed."
- No detail on which validation step failed
- No way to provide private key through the dialog

Workaround (proven 2026-05-16): SSH direct file replace, bypassing the UI.

```
# On Fleet appliance (lcm.lab.local) as root:
TS=$(date +%Y%m%d-%H%M%S)
cp /opt/vmware/vlcm/cert/server.crt /opt/vmware/vlcm/cert/server.crt.bak-$TS
cp /opt/vmware/vlcm/cert/server.key /opt/vmware/vlcm/cert/server.key.bak-$TS
cp /opt/vmware/etc/lighttpd/server.pem /opt/vmware/etc/lighttpd/server.pem.bak-$TS

install -m 600 -o root -g root /root/vcfa/vcfa.crt
/opt/vmware/vlcm/cert/server.crt
install -m 600 -o root -g root /root/vcfa/vcfa.key
/opt/vmware/vlcm/cert/server.key
cat /root/vcfa/vcfa.key /root/vcfa/vcfa.crt >
/opt/vmware/etc/lighttpd/server.pem.tmp
```

```
chmod 600 /opt/vmware/etc/lighttpd/server.pem.tmp
mv /opt/vmware/etc/lighttpd/server.pem.tmp /opt/vmware/etc/lighttpd/server.pem

nginx -t && systemctl reload nginx
systemctl restart lighttpd
```

Verify with a workstation-side TLS probe against `lcm.lab.local:443` — thumbprint should change to the new cert's SHA-1, and the SAN list should match exactly.

Alternative workaround (UI-only, not yet attempted in this lab): Generate the CSR through Fleet itself (which would keep the private key internally), sign the CSR externally with the appropriate SAN config, and paste the signed cert back through the same Import Certificate dialog. This matches Broadcom's intended UI workflow but is not documented in the dialog or in KB 411354's body.

Audit note: Searched Broadcom KB 2026-05-16 for `Import Certificate Fleet private key`, `Certificate replace operation for VCF Operations Fleet Management failed`, `VCF Operations Certificate Import dialog missing private key`. No KB matches. KB 411354 prescribes the cert SAN requirements but does not surface the CSR-must-come-from-Fleet-first requirement that this UI dialog enforces.

Discovered: 2026-05-16 during VCFA pre-deploy Phase 2 (cert regen). Full execution record: `VCFA-PreDeploy-Cleanup-CertReplace-2026-05-16.md`.

Issue #45: NSX `transport-node-collections` DELETE fails with error_code 26185 on any vLCM-flagged cluster, even when no vLCM image is set in vCenter

Problem: Attempting `DELETE /api/v1/transport-node-collections/<tnc-id>` to detach a TNP from a cluster — the documented path to force NSX re-prep — is rejected with `error_code 26185: "Transport Node Profile cannot be detached from vLCM enabled cluster."` This fires regardless of whether the vSphere cluster actually has a vLCM image defined.

Impact:

- The standard NSX recovery procedure ("detach TNP, fix TNP, reattach to re-prep all hosts") cannot be invoked via API in VCF-managed clusters
- Issue #43 and Issue #44 cannot be worked around via NSX un-prep
- All recovery is funneled into VCF Operations / SDDC Manager UI workflows that may not surface this specific scenario

Symptoms:

```
DELETE /api/v1/transport-node-collections/<tnc-id>
```

Response:

```
{
  "httpStatus": "BAD_REQUEST",
  "error_code": 26185,
  "module_name": "service-installation-fabric",
  "error_message": "Transport Node Profile cannot be detached from vLCM enabled
cluster."
}
```

Root cause investigation:

The gating flag is `origin_properties.lifecycleManaged = true` on the NSX-side compute-collection record:

```
GET /api/v1/fabric/compute-collections/<cm-id>:<cluster-moref>
```

returns (excerpt):

```
{
  "external_id": "<cm-id>:<cluster-moref>",
  "origin_type": "VC_Cluster",
  "origin_properties": [
    {"key": "lifecycleManaged", "value": "true"},
    ...
  ]
}
```

However, on the vCenter side, the same cluster's `ClusterConfigInfoEx.DesiredImageSpec` is **null** (no vLCM image set):

```
$cl = Get-Cluster -Name <cluster-name>
$cl.ExtensionData.ConfigurationEx.DesiredImageSpec   # returns null
```

So the lifecycle gate on NSX side is coming from **VCF's cluster-registration metadata**, not from vSphere's image-lifecycle subsystem. There is no documented API to flip this flag without going through the full VCF cluster lifecycle management workflow.

Workarounds attempted: None successful. The flag cannot be edited via NSX manager API (no documented endpoint), cannot be edited via vCenter API (it's NSX-side metadata), and the VCF UI workflow for this is not documented.

Audit note: Searched Broadcom KB 2026-05-16 for `error_code 26185`, `Transport Node Profile cannot be detached`, `vLCM enabled cluster`, `lifecycleManaged true compute collection`. No KB matches.

Discovered: 2026-05-16 during NSX cleanup work executed after VM migration was complete (Issue #43 and #44 had already been observed; this was the third independent gate that fired on the same orphan vDS scenario).

---

13. VCSA Recovery — May 2026

Issue #41: Host Client OVA deploy populates `vAppConfig` via alternate transport — `vim-cmd` shows null even on success

Problem: When VCSA 9.0.x is deployed via the ESXi Host Client UI ("Virtual Machines → Create/Register VM → Deploy from OVF/OVA"), the OVF properties (IP, hostname, DNS, root password, SSO config, etc.) ARE applied to the VM, but they do NOT populate the canonical `vAppConfig` field on the VM's ConfigInfo. `vim-cmd vmsvc/get.config <vmid>` returns `vAppConfig = (vim.vApp.VmConfigInfo) null` for a deploy that actually succeeded.

Impact: Troubleshooters seeing `vAppConfig=null` may incorrectly conclude the deploy failed (especially after a prior ovftool incident with the same symptom — see Issue #29 and Issue #42). Premature redeploy attempts waste 30+ minutes and risk data loss if the partially-booted appliance is destroyed before its OVF-environment ISO finishes processing.

Symptoms:

- `vim-cmd vmsvc/get.config <vmid>` returns `vAppConfig = (vim.vApp.VmConfigInfo) null`
- `vim-cmd vmsvc/get.guest <vmid>` returns `hostName = <unset>, ipAddress = <unset>` *for the first 5–15 minutes after power-on* (VMware Tools hasn't yet picked up post-firstboot state)
- The VM IS running and processing first-boot scripts (RPM installation visible on console)
- After ~15–25 minutes the appliance comes up with the correct IP, hostname, and identity that were entered in the wizard

Root cause: Host Client uses an OVF environment transport that is NOT the standard `VmConfigInfo.vAppConfig` field. Most likely an OVF-environment ISO mounted as CD-ROM (`/dev/sr0` from inside the guest) and consumed by the appliance's first-boot script directly. The VMX may also carry `guestinfo.*` ExtraConfig keys that `vim-cmd vmsvc/get.config` does not surface in the `vAppConfig` block.

How to identify a SUCCESSFUL Host Client OVA deploy (despite `vAppConfig=null`):

```
# ESXi shell (SSH) — verify the appliance is doing real work
# 1. Tools is reported "toolsOk", power state "poweredOn"
vim-cmd vmsvc/get.summary <vmid> | grep -E "toolsStatus|powerState|guestFullName"

# 2. Wait until first-boot finishes, then check the IP arrived
vim-cmd vmsvc/get.guest <vmid> | grep -E "hostName|ipAddress"

# 3. From workstation: ping the configured IP — definitive success signal
ping <configured-ip>     # answers when network stack is up
```

```
# PowerShell on workstation
Test-Connection -ComputerName 192.168.1.69 -Count 4
Test-NetConnection -ComputerName vcenter.lab.local -Port 5480  # Stage 2 VAMI
port
```

Resolution — do NOT panic-redeploy on `vAppConfig=null` alone:

```
1. Confirm the VM is powered on and Tools status is "toolsOk".
2. Wait the full first-boot window: ~15–25 minutes after power-on.
3. Check whether the appliance pings on the configured IP.
4. If YES: deploy succeeded. Continue to Stage 2 at https://<fqdn>:5480.
5. If NO after 25 min: THEN investigate. Likely OVF properties truly were dropped
(see Issue #42).
```

Audit note: Searched Broadcom KB 2026-05-13 for "Host Client OVA vAppConfig null", "VCSA Host Client deploy properties missing", "vim-cmd get.config vAppConfig null OVA". No KB documents the OVF-environment-transport difference between Host Client UI and `vim-cmd vmsvc/get.config` field reporting. Gap confirmed.

Discovered: 2026-05-13 during vCenter redeploy following the destroyed-vCenter scenario in `VCF9-Management-Plane-Loss-RCA-2026-05-01.md` §7.3. Distinct from Issue #29 (ovftool 5.0 actually-drops vApp properties) —

Issue #41 is a *reporting* gap (deploy succeeded, vim-cmd misreports state); Issue #29 is a *deployment* failure (ovftool truly drops the properties).

Issue #42: VCSA UI Installer 9.0 cannot deploy onto an existing vSAN datastore (KB 389238 — documented by Broadcom but routinely missed in recovery runbooks)

Note: This issue IS documented by Broadcom in KB 389238. It is included in this otherwise-undocumented-only reference because it is **routinely missed** in standard vCenter recovery procedures (including the `VCF9-Host-vCenter-Rebuild-Plan.md` v1 §6.2 / §7.3 which incorrectly directed users to the ISO installer UI before this discovery on 2026-05-13).

Problem: Both deployment paths offered by the VCSA 9.0.x UI Installer (`vcsa-ui-installer\<platform>\installer.exe`) on the "Select datastore" step fail when the target ESXi host has only a vSAN datastore visible:

Radio button	Failure mode
"Install on an existing datastore accessible from the target host"	Broadcom KB explicitly: "*Cannot use existing vSAN datastore for installation*"
"Install on an existing vSAN datastore and claim additional disks"	Stage 1 may complete, then Stage 2 fails at "*Install - Stage 2: vCenter Server setup is in progress*" per KB 389238. Additionally, Stage 1 validation can hang for 10–15 minutes on `vim.host.VsanSystemEx.GetVsanRuntimeInfo` with internal SSL-verify errors against `/vsanperf` if any local SSL certs are stale (see also Issue #41)

Impact:

- vCenter recovery on an esxi host whose only storage is the cluster vSAN datastore appears to be blocked
- Misled by the installer's UI (which presents the vSAN datastore in the dropdown without warning), operators spend hours debugging cert and validation issues that are actually symptoms of an unsupported deploy mode

Symptoms:

- Stage 1 "Validating" spinner runs 10+ minutes with no progress; hostd.log shows `SSLVerifyException` against `localhost.localdomain:80 /vsanperf` from the installer's API calls
- Or Stage 1 succeeds but Stage 2 hangs / fails

Resolution — bypass the VCSA UI Installer entirely; use Host Client OVA upload:

```
1. Mount the VCSA ISO (Windows: double-click in Explorer). Note the drive letter.
2. Locate the OVA file: <drive>:\vcsa\VMware-vCenter-Server-Appliance-
<version>_OVF10.ova (~8.85 GB for 9.0.1)
3. Open the target ESXi Host Client in a browser: https://<esxi-fqdn>/ui/
```

```
4. Login as root.
5. Virtual Machines → Create/Register VM → Deploy a virtual machine from an OVF or
OVA file → Next.
6. Enter VM name (e.g., "vcenter"). Drag/drop or browse to the .ova file.
7. Select the vSAN datastore (Host Client doesn't run the installer's vSAN-
bootstrap check — accepts vSAN normally).
8. Configure Network Mappings (e.g., pg-mgmt-ephemeral), Disk provisioning Thin,
Deployment Type tiny/small/etc.
9. Fill OVF properties:
   Networking: ipv4 / static / IP / prefix / gateway / DNS / FQDN
   Networking Properties: Domain Name, Domain Search Path
   SSO Configuration: vsphere.local / Default-Site / administrator / <password>
   System Configuration: root password, Enable SSH=True, NTP server
   Miscellaneous: CEIP=False
   Upgrade Configuration: LEAVE ENTIRELY BLANK (these are for migration, not fresh
install)
10. Power on → first-boot ~15–25 min → Stage 2 wizard at https://<fqdn>:5480
```

This is the path used in the 2026-05-01 management plane loss recovery — see `VCF9-Management-Plane-Loss-RCA-2026-05-01.md` §7.3 line 426.

Audit note: Documented by Broadcom (KB 389238), but the user-authored `VCF9-Host-vCenter-Rebuild-Plan.md` v1 lines 295–297 incorrectly prescribed "Deploy a new vCenter Server appliance via the ISO installer UI" without referencing KB 389238. Runbook corrected 2026-05-13 to direct readers to KB 389238 + Host Client OVA path. Included here for traceability and as a warning to future operators encountering similar runbook gaps.

Discovered: 2026-05-13 during vCenter redeploy after operator went through full Stage 1 of UI installer and hit `GetVsanRuntimeInfo` SSL-verify hang. Cancelled wizard after Broadcom KB lookup revealed the limitation.

---

14. Fleet Management Offline Depot — May 2026

Issue #47: Fleet Management offline depot metadata zip references superseded 9.0.2.0 patch binary — Broadcom only ships 9.0.2.0100, causing `LCMPATCHUPDATE16002` on patch apply

Problem: The VCF offline depot metadata zip (`vcf-9.0.2.0-offline-depot-metadata.zip` , downloaded from Broadcom Support Portal as of 2026-05-26) contains a `productVersionCatalog.json` that references the Fleet Management patch binary `VCF-OPS-Lifecycle-Manager-9.0.2.0.25137839.patch` . However, Broadcom has superseded that binary with `VCF-OPS-Lifecycle-Manager-9.0.2.0100.25362033.patch` (released April 27, 2026) and removed the 9.0.2.0 binary from the downloads portal. The metadata zip was never updated to match.

This creates a version mismatch chain:

Component	Version it references	Version that actually exists
`productVersionCatalog.json` (metadata zip)	`9.0.2.0.25137839`	n/a (removed from portal)
Patch binary on Broadcom downloads	n/a	`9.0.2.0100.25362033`
Internal `patchProductRegistry.json` (extracted from .patch)	`9.0.2.0100`	(correct — matches actual binary)

When the operator renames the 9.0.2.0100 binary to match the catalog's expected filename, Fleet downloads it successfully and even passes checksum validation (the download checksum is validated against the patch's internal manifest, not the catalog). But on patch apply, Fleet's `PatchUpdateTask` constructs the internal content URL from the `patchProductRegistry.json` version (`9.0.2.0100`) while the `vm_cr_contentdownloadurl` table stores the blob under the catalog's version (`9.0.2.0`). Result: `NullPointerException` at `PatchUpdateTask.createLocalFile()` → `LCMPATCHUPDATE16002 "Error while creating local patch file"` → `"Patch validation failed"` .

Impact:

- Fleet Management cannot be upgraded from 9.0.1 to 9.0.2.0100 via the standard UI patch workflow when using offline depot
- Without Fleet at 9.0.2.0100, VCF Automation 9.0.2 cannot be deployed (Fleet's policy layer caps deployable product versions to its own version)
- Any operator using offline depot with the current metadata zip hits this on the first patch attempt

Symptoms:

```
Error Code: LCMPATCHUPDATE16002
Failed to apply VCF Operations Fleet Management patch.
Error while creating local patch file

com.vmware.vrealize.lcm.commons.patch.exceptions.PatchException:
  Error while creating local patch file
  at PatchUpdateTask.initParams(PatchUpdateTask.java:336)
```

Followed immediately by:

```
Patch validation failed.
com.vmware.vrealize.lcm.commons.patch.exceptions.PatchException:
  Patch validation failed.
  at PatchUpdateTask.execute(PatchUpdateTask.java:109)
```

In `/var/log/vrlcm/vmware_vrlcm.log`, the critical sequence is:

```
LCM patch uRL: https://lcm.lab.local/repo/productPatchRepo/patches/vrlcm/9.0/
  VCF-OPS-Lifecycle-Manager-Appliance-9.0.2.0100.patch
ContentDownloadController -- Error occurred when processing the download content.
PatchUpdateTask -- Error occurred in downloading patch from content repo
java.lang.NullPointerException: null
  at PatchUpdateTask.createLocalFile(PatchUpdateTask.java:199)
```

The URL Fleet requests (`...9.0.2.0100.patch`) does not match the URL stored in `vm_cr_contentdownloadurl` (`...9.0.2.0.patch`).

Root cause: Broadcom updated the Fleet Management patch binary from build 25137839 to 25362033 (version 9.0.2.0 → 9.0.2.0100) on their downloads portal but did not update the offline depot metadata zip. The metadata zip filename is unchanged (`vcf-9.0.2.0-offline-depot-metadata.zip`), its file size is unchanged (188,858 bytes), and the `sequenceNumber` is unchanged (33). There is no indication on the portal that the metadata is stale relative to the available binaries.

Resolution — single-row database fix on Fleet appliance:

The patch binary IS correctly downloaded and stored in Fleet's internal content repository (blob UUID `7a2bdca3-...` , 1.2 GB). The only problem is the URL mapping. Fix by updating one row in `vm_cr_contentdownloadurl` :

```
-- SSH to Fleet appliance, then:
cd /tmp && sudo -u postgres /opt/vmware/vpostgres/current/bin/psql -d vrlcm

-- Verify current (broken) state:
SELECT vmid, url, content FROM vm_cr_contentdownloadurl
WHERE url LIKE '%Lifecycle-Manager-Appliance-9.0.2%';
-- Returns: .../VCF-OPS-Lifecycle-Manager-Appliance-9.0.2.0.patch

-- Fix the URL to match what patchProductRegistry expects:
UPDATE vm_cr_contentdownloadurl
SET url = '/productPatchRepo/patches/vrlcm/9.0/VCF-OPS-Lifecycle-Manager-
Appliance-9.0.2.0100.patch'
WHERE url = '/productPatchRepo/patches/vrlcm/9.0/VCF-OPS-Lifecycle-Manager-
Appliance-9.0.2.0.patch';

-- Verify (should show 9.0.2.0100 in the URL now):
```

```
SELECT vmid, url, content FROM vm_cr_contentdownloadurl
WHERE url LIKE '%Lifecycle-Manager-Appliance-9.0.2%';
```

```
# PowerShell on workstation (via Invoke-VMScript):
$fleet = Get-VM 'fleet'
Invoke-VMScript -VM $fleet -GuestUser 'root' -GuestPassword '<password>' -
ScriptType Bash -ScriptText @'
cd /tmp && sudo -u postgres /opt/vmware/vpostgres/current/bin/psql -d vrlcm -c "
UPDATE vm_cr_contentdownloadurl
SET url = '/productPatchRepo/patches/vrlcm/9.0/VCF-OPS-Lifecycle-Manager-
Appliance-9.0.2.0100.patch'
WHERE url = '/productPatchRepo/patches/vrlcm/9.0/VCF-OPS-Lifecycle-Manager-
Appliance-9.0.2.0.patch';"
'@
```

After the update, verify the content controller resolves the new URL:

```
curl -sk -o /dev/null -w '%{http_code}' \
  "https://localhost/lcm/repo/productPatchRepo/patches/vrlcm/9.0/VCF-OPS-
Lifecycle-Manager-Appliance-9.0.2.0100.patch"
# Should return 200
```

Then retry: **Settings → System administration → System patches → Install patch → fleet management → Install**.

Audit note: Searched Broadcom KB 2026-05-26 for `LCMPATCHUPDATE16002`, `offline depot metadata 9.0.2.0100`, `Fleet Management patch validation failed createLocalFile`, `productVersionCatalog 9.0.2 stale`. No KB documents the metadata-to-binary version mismatch for the 9.0.2.0 → 9.0.2.0100 supersession, or the `vm_cr_contentdownloadurl` fix. KB 432806 ("Configure Offline Depot with binary version") covers offline depot setup but does not address the version-mismatch scenario. KB 422542 covers the Local Path depot type but not URL-based offline depot.

Discovered: 2026-05-26 during Fleet Management upgrade from 9.0.1 to 9.0.2 in preparation for VCF Automation 9.0.2 deployment. Patch binary downloaded from Broadcom Support Portal Packlist ID 537849 was build 25362033 (9.0.2.0100); metadata zip from the same portal still referenced build 25137839 (9.0.2.0). Root cause identified by tracing the `PatchUpdateTask` code path through Fleet's internal content repository tables.

---

15. Originally Discovered Undocumented — Now Documented by Broadcom (9 issues)

The following 9 issues were independently discovered during VCF 9.0 lab operations and tracked in this document before Broadcom published KB articles or TechDocs covering them. They were removed in the v6.0 audit (May 11, 2026) and moved to `VCF_Troubleshooting_Handbook` Section 12. They are restored here for completeness and to maintain an accurate total discovery count.

> *Each entry below includes the Broadcom reference that now documents it. Full verified procedures remain in `VCF_Troubleshooting_Handbook` Section 12.*

Issue #48 (formerly #14): NSX Manager Certificate SAN Validation Errors

Problem: NSX Manager certificate operations fail with SAN mismatch errors when the cert was generated with incorrect or incomplete Subject Alternative Names.

Now documented: KB 413265

Discovered: January–February 2026 during initial VCF 9.0.1 deployment.

Issue #49 (formerly #15): NSX Manager Has Two Separate Trust Stores

Problem: NSX Manager maintains two independent trust stores. Importing a certificate into one does not propagate to the other, causing trust validation failures that appear inconsistent.

Now documented: KB 316056 (updated)

Discovered: January–February 2026 during initial VCF 9.0.1 deployment.

Issue #50 (formerly #16): Fleet Management Cert Generator Produces Wrong SANs

Problem: Fleet Management's "Generate self-signed certificate" wizard option produces a cert whose SAN entries don't match the node FQDN/IP, causing deployment precheck failures with "The hosts in the certificate doesn't match with the provided/product hosts."

Now documented: KB 412322, KB 421072

Discovered: January–February 2026 during initial VCF 9.0.1 deployment. Workaround: generate cert manually with OpenSSL.

Issue #51 (formerly #19): VCF Operations 9.x Adapter Log Paths Changed

Problem: Legacy VCF Operations adapter troubleshooting guides reference log paths that don't exist on 9.x appliances. The correct path is `/storage/log/vcops/log/adapters/`.

Now documented: TechDocs — Location of Log Files

Discovered: February–March 2026 during initial deployment and crash recovery.

Issue #52 (formerly #25): vCenter Migration Wizard Cannot Thin-Provision to vSAN

Problem: vCenter's storage migration wizard fails or produces thick-provisioned results when migrating VMs to vSAN. The only reliable path is `vmkfstools -i <src> <dst> -d thin` per disk directly on the ESXi host.

Now documented: KB 389960

Discovered: February–March 2026 during crash recovery. SDDC Manager went from 914 GB allocated to ~108 GB actual after manual thin clone.

Issue #53 (formerly #28): VDT (VCF Diagnostic Tool) Not Pre-Installed on SDDC Manager

Problem: VDT is not included on the SDDC Manager appliance. Must be downloaded from Broadcom and manually installed before running diagnostics.

Now documented: KB 344917

Discovered: March 2026 during crash recovery.

Issue #54 (formerly #29): Suite-API Auth Header Changed to `OpsToken`

Problem: VCF Operations Suite-API now uses `Authorization: OpsToken <token>` instead of the legacy `Authorization: vRealizeOpsToken <token>`. Legacy format still works but is deprecated.

Now documented: TechDocs — Acquire an Authentication Token

Discovered: March 2026 during crash recovery.

Issue #55 (formerly #36): Photon OS 5 Static IP Lost on Reboot (Desired State Model)

Problem: Manually editing `/etc/systemd/network/10-eth0.network` on VCF Photon OS 5 appliances doesn't persist — the appliance reads vApp properties from vCenter on every boot and overwrites the network config. Must set IP via vApp properties in vCenter, not in the guest OS.

Now documented: KB 425535

Discovered: April 2026 during Aria Suite / VCFA deployment.

Issue #56 (formerly #46): Fleet Management Binary Management Page Empty with Local Path Depot

Problem: When offline depot is configured using "Local Path" mode, the Binary Management page shows no installation packages even though binaries exist at the configured path. The metadata files must be staged at `/data/temp/PROD/metadata/` on the Fleet appliance, then the depot settings must be re-saved to trigger a re-scan.

Now documented: KB 422542, TechDocs — Depot Configuration

Discovered: April 2026 during Aria Suite / VCFA deployment.

---

16. Quick Reference: All 56 Issues

#	Issue	Category	Severity	Fix Summary
1	PostgreSQL schema unmapped	Database	Critical	Map via `information_schema` queries
2	Credential cascade failure	Database	Critical	6-step DB repair (Issue #4)
3	API can't cancel stuck tasks	Database	Critical	`UPDATE task_metadata SET resolved = true`
4	6-step repair must be in sequence	Database	Critical	Full procedure with all SQL commands
5	PostgreSQL requires `-h 127.0.0.1`	Database	Medium	Always use `-h 127.0.0.1` flag
6	Column naming inconsistencies	Database	Medium	`state` not `status`, `resolved` boolean
7	PostgreSQL password not discoverable	Database	High	`pg_hba.conf` trust auth workaround
8	NSX needs 32GB RAM (not 16GB)	NSX	High	Set VM to 32GB RAM / 6 vCPU
9	Boot storm load >100 is normal	NSX	High	Wait 30-60 min after power-on
10	More vCPU makes it worse	NSX	Medium	Keep at 6 vCPU, increase RAM instead

11	Services need 10-15 min	NSX	Medium	Wait before API calls; use monitoring loop
12	DNS/NTP via CLI only	NSX	Medium	`set name-servers`, `set ntp-servers`
13	TEP on vmk0 (new in 9.0)	NSX	Low	Select "Use VMkernel Adapter" in Transport Profile
14	Logs cert generator wrong SANs	Certs	High	Full OpenSSL cert generation procedure
15	Shell can't handle PEM escaping	Certs	Medium	Python script to build JSON payload
16	JRE path changed	VCF Ops	Medium	`/usr/java/jre-vmware-17/`
17	Two separate NSX adapters	VCF Ops	Medium	Configure both VCF and Aria Admin adapters
18	Credential ROTATE broken for NSX	VCF Ops	High	Uncheck system managed, set manually
19	SSH enable via Admin UI only	VCF Ops	Medium	`https://<host>/admin/` > SSH > Enable
20	Health adapter fails silently	VCF Ops	High	Full appliance reboot required
21	vhv.enable ghost setting	Infra	Medium	Add `vhv.enable = "FALSE"` to VMX
22	Hot vMotion fails nested	Infra	Medium	Power off VM, cold migrate, power on
23	Permissions API single object	Suite-API	High	`{"roleName":"Administrator","allowAllObjects":tru`
24	Super admin empty roles	Suite-API	Low	By design — no action needed
25	domainmanager port 7200 HTTP	SDDC	Medium	Use `http://localhost:7200` not https
26	NSX credential fields uppercase	Suite-API	Medium	`USERNAME` and `PASSWORD` (not USER)
27	Gemfire cache needs 5-10 min	VCF Ops	Medium	Wait after init; use monitoring loop
28	HSQLDB reset after crash	VCF Ops	Critical	Full sed/restart/init procedure

29	OVFTool silently drops InstanceId properties	OVFTool	High	Set vApp properties in vCenter after deploy
30	`nmctl` not available on VCF Ops appliance	Networking	Medium	Use vApp properties or `ip addr add`
31	VMware API effectiveMemory in MB, totalMemory in bytes	vCenter API	Critical	Multiply effectiveMemory by 1048576
32	NSX http service doesn't auto-start after reboot	NSX	High	Run `start service http` from NSX CLI
33	LCM UI creates orphaned UI_INPROGRESS records	Fleet Mgmt	Critical	Full database cleanup required (6 tables)
34	LCM creates both `vra` AND `vmsp` product entries	Fleet Mgmt	Critical	Must clean both product IDs
35	Parallel deployment check is in VCF Ops frontend, not LCM	Fleet Mgmt	Critical	Restart vmware-vcops-web AND clean LCM DB
36	VCF Automation ipPool must be in nodes[0].properties	LCM API	Critical	Not in product properties
37	Suite-API auth header changed to OpsToken	Suite-API	High	Use `Authorization: OpsToken` not vRealizeOpsToken
38	vSAN OSA `.vmx virtualSSD` for nested (not SATP)	vSAN/Nested	High	Add `sata0:3.virtualSSD = "1"` etc to outer `.vmx`
39	`data-move` summary excludes Decommissioning intent	vSAN	Medium	Use `esxcli vsan debug resync summary get` bytes
40	`clusterAdmin runReplica` exit:2 blocks MM	Cluster Store	High	Manual `esxcli system maintenanceMode set --enable true`

41	Host Client OVA deploy `vAppConfig=null` is misleading	VCSA / OVA	Medium	Wait full 15–25 min firstboot, ping configured IP — do not panic-redeploy
42	VCSA UI Installer 9.0 cannot use existing vSAN (KB 389238)	VCSA / vSAN	High	Bypass installer; deploy via Host Client OVA upload at `https://<esxi>/ui/`
43	NSX `?action=clear` ownership fails 8012 when vCenter doesn't have the orphan vDS UUID	NSX vDS	High	No API recovery; full un-prep+re-prep via VCF UI; SR Broadcom
44	NSX per-host TN PUT fails 9560 — TZ relocation single-transaction guard	NSX vDS	High	Same — un-prep+re-prep via UI; two-PUT workaround not documented
45	NSX TNC DELETE fails 26185 — vLCM-enabled cluster, even with no vLCM image set in vCenter	NSX vDS / vLCM	High	`lifecycleManaged=true` flag is NSX-side; no API to flip; UI workflow only
46	VCF Ops Import Certificate UI has no Private Key field — expects pre-generated Fleet CSR	Cert / Fleet UI	High	Workaround: SSH direct replace at `/opt/vmware/vlcm/cert/server.{crt,key}` + `/opt/vmware/etc/lighttpd/server.pem`, reload nginx + restart lighttpd
47	Offline depot metadata references superseded 9.0.2.0 patch — Broadcom only ships 9.0.2.0100	Fleet Depot	Critical	`UPDATE vm_cr_contentdownloadurl SET url = '...9.0.2.0100.patch'` to fix internal path mismatch
	Now Documented by Broadcom (9 issues)			
48	NSX Manager certificate SAN validation errors	NSX / Certs	High	KB 413265
49	NSX Manager has two separate trust stores	NSX / Certs	High	KB 316056

50	Fleet Management cert generator produces wrong SANs	Fleet Mgmt	High	KB 412322, KB 421072
51	VCF Operations 9.x adapter log paths changed	VCF Ops	Medium	TechDocs
52	vCenter migration wizard cannot thin-provision to vSAN	vCenter / vSAN	Medium	KB 389960
53	VDT not pre-installed on SDDC Manager	SDDC Manager	Medium	KB 344917
54	Suite-API auth header changed to `OpsToken`	Suite-API	High	TechDocs
55	Photon OS 5 static IP lost on reboot (Desired State model)	Photon OS	High	KB 425535
56	Binary Management page empty with Local Path depot	Fleet Mgmt	High	KB 422542

Numbering note: Issues #1–47 are currently undocumented by Broadcom. Issues #48–56 were originally discovered undocumented and are now covered by Broadcom KBs/TechDocs. Total discoveries: 56. Issues #1–40 numbered in v6.0 post-audit; #41–42 added in v6.1 from May 13 vCenter recovery; #43–45 added in v6.2 from May 15–16 NSX orphan vDS cleanup work. Nine prior entries from v5.x were moved to `VCF_Troubleshooting_Handbook` after Broadcom shipped KB articles for them — see Revision History v6.0 for the mapping of old numbers to current Broadcom KBs.

---

17. Document Information

Field	Value
Document Title	VCF 9.0 Undocumented Issues & Discoveries Reference
Version	7.0
Author	Virtual Control LLC
Date Created	March 15, 2026

Last Updated	May 26, 2026
Total Discoveries	56 (47 still undocumented + 9 now documented by Broadcom)
Environment	VMware Cloud Foundation 9.0.1 / Nested Lab
Issues #1–15	Discovered during initial VCF 9.0.1 deployment (Jan–Feb 2026)
Issues #16–22	Discovered during initial deployment + crash recovery (Feb–Mar 2026)
Issues #23–28	Discovered during Windows Update crash recovery (March 2026)
Issues #29–30	Discovered during VCF Operations redeployment (April 2026)
Issues #31–37	Discovered during Aria Suite / VCFA deployment (April 14–16, 2026)
Issues #38–40	Discovered during esxi02/esxi03 vSAN disk-group rebuild (May 9–11, 2026)
Issues #41–42	Discovered during vCenter recovery + redeploy (May 13, 2026)
Issues #43–45	Discovered during NSX orphan vDS cleanup attempts after VCF management plane VM migration (May 15–16, 2026); Broadcom SR drafted: `Broadcom-SR-NSX-Orphan-vDS-Cleanup-2026-05-16.md`
Issue #47	Discovered during Fleet Management offline depot upgrade 9.0.1 → 9.0.2.0100 (May 26, 2026)
Removed in v6.0 audit	9 prior entries (now in Broadcom KBs — moved to VCF_Troubleshooting_Handbook). Mapping: old #14→KB 413265, #15→KB 316056, #16→KB 412322/421072, #19→TechDocs, #25→KB 389960, #28→KB 344917, #29→TechDocs, #36→KB 425535, #46→TechDocs/KB 422542

Revision History

Version	Date	Change
5.0	April 16, 2026	Added issues #39–46 (Aria Suite deployment discoveries).
5.1	May 7, 2026	Option B PowerShell-sibling pass: every bash block now has a paste-ready PowerShell equivalent (workstation-side flows: REST API, OVFTool). Appliance-only blocks (psql, systemctl, keytool against `/etc/`, file edits in `/storage/` or `/etc/systemd/network/`) and ESXi-shell blocks (`vmkfstools`, VMX edits) intentionally skipped per Option B scope. Top-of-doc callout links to `../POWERSHELL-TRANSLATION-REFERENCE.md`.
6.0	May 11, 2026	**Major audit + restructure + sequential renumber.** (1) Audited all 46 prior issues against current Broadcom KB state (May 2026); removed 9 issues now documented by Broadcom and moved their content to `VCF_Troubleshooting_Handbook`. Old → KB mapping: #14 → KB 413265 (NSX cert SAN); #15 → KB 316056 updated (two trust stores); #16 → KB 412322 / KB 421072 (Fleet cert generator); #19 → TechDocs (adapter log paths); #25 → KB 389960 (thin to vSAN); #28 → KB 344917 (VDT download); #29 → TechDocs (vRealizeOpsToken); #36 → KB 425535 (Photon OS 5 Desired State); #46 → TechDocs + KB 422542 (Local Path depot). (2) Added 3 new issues from May 2026 vSAN disk-group rebuild work: vSAN

		OSA `.vmx virtualSSD` for nested ESXi (not SATP rule); `data-move` summary excludes Decommissioning intent; `clusterAdmin runReplica` exit:2 blocks MM completion. (3) **Renumbered all remaining 40 issues sequentially 1–40** for readability (no more numbering gaps). Cross-references in body text auto-updated. (4) Restructured TOC + categories: new sections 9 (Aria Suite / LCM / VCF Automation) and 10 (vSAN & DVS Edge Cases — May 2026); Quick Reference moved to section 11; Document Information to section 12. (5) Updated Summary by Category table and Discovery Timeline to reflect new sequential ranges. New total: 40 verified-undocumented issues.
6.1	May 13, 2026	Added Section 11 "VCSA Recovery — May 2026" with two new issues from vCenter redeploy work: Issue #41 (Host Client OVA deploy `vAppConfig=null` is misleading — deploy succeeded, vim-cmd misreports the canonical field; undocumented) and Issue #42 (VCSA UI Installer 9.0 cannot use existing vSAN datastore — documented by Broadcom KB 389238 but routinely missed in user-authored recovery runbooks; included for traceability/warning). Renumbered Quick Reference to Section 12 and Document Information to Section 13. Total: 42.
6.2	May 16, 2026	Added Section 12 "NSX 9.0 vDS Replacement / vLCM Lifecycle Edge Cases" with three new issues discovered during NSX orphan vDS cleanup after VCF management plane VM migration: Issue #43 (NSX `?action=clear` ownership fails 8012 when vCenter doesn't have the orphan vDS UUID — API enum locked to `[set, clear]`, no `force=true`); Issue #44 (per-host TN PUT fails 9560 — TZ single-transaction relocation guard; workaround procedure not documented); Issue #45 (TNC DELETE fails 26185 — vLCM-enabled cluster gate triggered by NSX-side `lifecycleManaged=true` flag even when vCenter has no actual vLCM image set). All three escalated to Broadcom via SR draft `Broadcom-SR-NSX-Orphan-vDS-Cleanup-2026-05-16.md`. Renumbered VCSA Recovery to Section 13, Quick Reference to Section 14, Document Information to Section 15. Total: 45.
6.4	May 26, 2026	Added Section 14 "Fleet Management Offline Depot — May 2026" with Issue #47: offline depot metadata zip references superseded `9.0.2.0.25137839` Fleet patch binary while Broadcom only ships `9.0.2.0100.25362033`; `PatchUpdateTask` fails with `LCMPATCHUPDATE16002` due to `vm_cr_contentdownloadurl` path mismatch between catalog-derived URL (`9.0.2.0`) and `patchProductRegistry`-derived URL (`9.0.2.0100`). Fix: single-row UPDATE in Fleet Postgres. Renumbered Quick Reference to Section 15, Document Information to Section 16. Total: 47.
7.0	May 26, 2026	**Restored 9 previously removed issues** as Section 15 "Originally Discovered Undocumented — Now Documented by Broadcom." These 9 issues (#48–56) were removed in v6.0 after Broadcom published KB articles covering them. Restored to maintain a complete discovery tally. Each entry includes the Broadcom KB/TechDocs reference. Renumbered Quick Reference to Section 16, Document Information to Section 17. **Total: 56** (47 still undocumented + 9 now documented).

This document is part of the VCF 9.0 Lab Documentation Suite:

- **VCF9-Master-Bible.pdf** — Complete VCF 9.0 reference guide
- **VCF_Troubleshooting_Handbook.pdf** — General troubleshooting procedures (now hosts the 9 issues moved out in v6.0 audit)
- **vSAN-OSA-Disk-Group-Rebuild-Handbook-2026-05-08.pdf** — Source of issues #47–48 (verified procedure)
- **VCFA-Deployment-Fix-Runbook-2026-05-08.pdf** — Source of issues #47–49 context
- **VCF-Environment-Health-Check.pdf** — Reusable environment health check
- **VCF-SDDC-Manager-API-Handbook.pdf** — SDDC Manager API reference